Client Metadata Outreach Prioritizer — MVP validation

Team T5 | November 9, 2026 | MVP validation

This app helps a MetaBrainz outreach coordinator decide which ListenBrainz submission-client maintainers to investigate for a recording-identifier pilot. Missing identifiers can signal an opportunity to investigate; they do not prove who caused a gap or what intervention would work. The app organizes aggregate evidence for human review and sends no messages.

Live app: Client Metadata Outreach Prioritizer. Deployment uses the `case-live-evidence` branch and root `streamlit_app.py`. The repository bundles the complete course sample of 21,276 events, with unused columns removed and account numbers consistently replaced. This is sampled submitted listening data, not all client users, a complete playback history, or the full raw dump. Counts, rates, eligibility, and rankings can differ from full-data results.

The five supplied individual cases appear in full in Section 2. Their original observations are historical evidence and may describe earlier app versions. The demo video is supplied separately by a teammate; its link must be included in the root README before hand-in.

1. Design

Users, research and the decision

The intended user is a coordinator allocating limited investigation effort before proposing a pilot to client maintainers. This role and workflow are a design scenario, not an interview with an actual MetaBrainz employee. The goal is a defensible shortlist and a reproducible comparison, not the highest missing-ID percentage in isolation.

The MetaBrainz ListenBrainz project's JSON Documentation distinguishes the software submitting listens from the media player and distinguishes submitted identifiers from server-resolved mappings. These distinctions support a design judgment: show submitted-ID gaps without blaming a client or claiming that a syntactically valid identifier identifies the correct recording. The project's Metadata API documentation documents text-based recording lookup and manual mapping. Those are alternatives to investigate alongside client outreach, not validated remedies for this sample. The course sampling methodology explains uniform random sampling and historical playback within a submission-day dump. Therefore the interface labels sample scope and uses playback dates rather than treating the dump date as the date of every listen.

Nora's case in Section 2 develops these sources and implications in a specific decision. No adoption study, maintainer interview or causal experiment was documented in these supplied cases. Current maintainer activity, willingness, costs and technical feasibility require evidence outside this dataset.

Why the implemented features fit

Feature	Decision served	Evidence or design reason
Shortlist and reach/volume weight	Identify candidates worth investigating	Distinct and affected accounts prevent event volume from being mistaken for people; sensitivity shows the value judgment in the weight.
Account floor and importer exclusion	Avoid basing a pilot on a tiny or inappropriate group	Exact classification is inspectable. Unknown types remain unverified; the floor is not an anonymity guarantee.
Absent/invalid-ID breakdown	Distinguish kinds of submitted-ID gaps	Different problems may need different investigation; neither category establishes client causation.
Client version and weekly evidence	Inspect concentration and observation scope	Unknown versions are disclosed. Unavailable weeks remain gaps, and edge weeks are marked partial.
Three scenario measures	Distinguish an approximate count from a threshold	Pano's 1% case requires 5 approximate additions but 6 minimum additions. Achieved coverage makes that mismatch visible.
Complete comparison plus individual briefing	Share a reproducible recommendation	All selected candidates and common settings travel together; users can still export one inspected client.
Portable sample and locked environment	Make the app usable by reviewers	A fresh environment and deployment need no sibling dataset folder or raw archive.

The score uses affected-account reach and unusable-ID volume after logarithmic normalization within the complete eligible population. Its explicit weight is a user preference, not a learned probability. The app deliberately does not predict pilot impact, infer maintainer activity, verify every UUID against MusicBrainz, send outreach, or expose event-level downloads. Those actions need additional evidence or authority. The deployed sample supports interaction and exact sample counts at modest size; it does not justify population market shares or current install-base estimates.

Earlier design notes proposed DuckDB and percentile ranking. The implemented app uses Arrow/Parquet batches, exact account sets with a spill path, and the explicit logarithmic score in Section 3. This validation describes the actual source and deployed behavior, not those earlier proposals.

Work beyond the starting build

The existing case identified CR-03 (threshold interpretation) and CR-04 (incomplete comparison export). The MVP preserves the original nearest-rounded measure and adds achieved coverage and minimum-to-reach counts. It adds complete CSV and Markdown comparison exports, while preserving individual-client exports. CR-02 adds absent/invalid columns to the shortlist. Version-gap ranking and contextual coverage address CR-05/06; all-client sensitivity and the account-distribution view address CR-07/08. The shortlist now labels its outreach status as unverified, addressing CR-09 without claiming an ownership verification that the data do not contain. The public sample launcher, portable run instructions, project dependency lock and redeployment-cache repair address reproducibility. These changes have dedicated regression checks and new live evidence, rather than being credited solely from a feature list.

2. Cases

Case coverage

The five supplied individual cases cover different people and decisions. They include a MetaBrainz screening analyst, an outreach coordinator comparing candidates, a Navidrome version investigator, a Pano maintainer making a support decision, and a coordinator verifying an unknown client label. Together they include difficult cases: a low target that exposes rounding, changing ranks under different priorities, version traffic that does not measure support burden, and a high-ranked label with unverified ownership. All five supplied PDFs are preserved as submitted; Rachel's PDF does not identify an author, so this document does not infer one.

Supplied case	Person and decision	Evidence and limitation
Tinyu: Nora	Whether the two-client comparison supports a pilot recommendation	Live Pano/Navidrome evidence, 1% threshold and complete-briefing concern; does not predict impact.
Leyna: Morgan	Whether Pano version traffic justifies keeping compatibility support	Live version evidence and official release/issue context; sample listens are not installations or support requests.
Zhongze: Alex	Which Navidrome version to investigate first	Live version counts and coverage, with official Navidrome/ListenBrainz context; a truncated version label is not a verified release identity.
Gigi: Ana	Which client merits the first diagnostic review under different weights	Live rank comparisons and documented sources; ranking stability at three weights is not a promise of causality or future stability.
Supplied Rachel case	Whether to add the unknown `listenbrainz` label to an outreach list	Live unknown classification and external documentation; a top score does not establish ownership or eligibility.

The source index below supplies clickable links for the preserved Rachel and Zhongze PDFs, whose original pages show URLs but do not contain clickable link annotations. It also points readers directly to the principal sources in the other cases:

- ListenBrainz, JSON documentation: client-submitted metadata, client/version fields, and server-resolved mappings.
- ListenBrainz, Core API: listen submissions; relevant to Rachel's eligibility and submission-context decision.
- Navidrome, scrobbling documentation: ListenBrainz integration; relevant to Alex's version investigation.
- Pano Scrobbler, releases and historical issue 488: release context and an example of a version-specific user report for Morgan's support decision.
- MusicBrainz, MusicBrainz Identifier: identifiers distinguish database entities, relevant to Ana's diagnosis.
- ListenBrainz, Metadata API: lookup and manual-mapping alternatives, relevant to Nora's comparison.

Each original PDF follows in full, including its source information and genuine live-app screenshots. The original screenshot values and change-request statuses document the app as observed on November 7–8; the subsequent MVP changes are summarized after the cases. A case may propose an investigation without asserting that the software caused a gap or that a maintainer has been contacted.

Nora's pilot decision: can this comparison support an outreach recommendation?

Tinyu Xie | DSO 576 | Individual case | November 7, 2026

Evidence note. Live-app evidence checked November 7, 2026. The public app opened in a separate browser session without signing in. All screenshots are genuine captures of that public app; independent calculations use the same portable sample.

Nora should use the Client Metadata Outreach Prioritizer to identify candidates for investigation, then ask maintainers about feasibility before recommending a pilot. She should request a corrected comparison briefing before recommending Pano Scrobbler over Navidrome: the hypothetical additions do not establish deliverable impact, a small target can be missed after rounding, and the current briefing does not carry both candidates' numerical evidence together.

1. The person and the decision

Nora is a fictional MetaBrainz outreach coordinator preparing a two-candidate pilot recommendation. She must decide whether the app's comparison provides sufficient evidence to recommend investigating Pano Scrobbler ahead of Navidrome, or whether to return the briefing for correction. She wants an auditable recommendation that distinguishes observed identifier gaps from hypothetical improvements. A misleading recommendation could direct limited outreach effort toward an unverified opportunity or promise a milestone the displayed number does not reach. This is a scenario-comparison and briefing decision, rather than a general highest-volume-client shortlist.

2. Her real situation and alternatives

ListenBrainz's JSON Documentation, published by the MetaBrainz ListenBrainz project, distinguishes client-submitted identifiers in *additional_info* from server-resolved *mbid_mapping*. It also distinguishes the submission client from the media player. These distinctions matter to Nora: missing submitted identifiers alone cannot identify which software component needs changing, and syntactically valid submitted IDs need not identify the correct recording.

The project's Metadata API documentation describes lookup by artist and recording name, optionally with a release name, and manual mapping submissions. Nora therefore has alternatives to a client outreach pilot: evaluate matching or mapping workflows, inspect example submissions, and discuss the submission path with maintainers.

Sources and supporting passages, read November 7, 2026:

- MetaBrainz ListenBrainz, JSON Documentation: “How mbid_mapping differs from additional_info” and “Client Metadata examples”: client-supplied IDs can be incorrect; the submission client and player can be different programs.
- MetaBrainz ListenBrainz, Metadata API: “GET /1/metadata/lookup/” and “POST /1/metadata/submit_manual_mapping/”: documents text-based lookup and manual recording mapping.
- USC DSO 576, How these samples were cut: describes uniform random sampling and explains why a submission-day dump includes historical playback dates. This supports the sample-scope limitation below.

3. The app in use

Live app: Client Metadata Outreach Prioritizer. Public deployment: team repository, branch *case-live-evidence*, entry point *streamlit_app.py*. The hosted sample retains every source sample event and the measurement fields, omits unused columns and user names, and consistently replaces account numbers. It is not full-population data.

Use sample mode; inclusive UTC playback dates 2005-02-13 through 2026-09-03; exclude classified importers; minimum distinct accounts 100; affected-account reach weight 50%. In the comparison selector choose navidrome and pano scrobbler. First set scenario target coverage to 75%; then test 1%. Inspect Pano Scrobbler in Client detail. Finally open Briefing with Navidrome inspected while both comparison clients remain selected.

Independently verified sample measures

Measure	Navidrome	Pano Scrobbler
Observed listens	889	520
Listens with a syntactically valid recording MBID	469	0
Submitted-MBID coverage	52.7559%	0%
Observed distinct accounts	719	360
Accounts with at least one unusable-ID listen	318	360
Hypothetical additions at a 75% target	198	390
Hypothetical additions at a 1% target	0	5

The underlying portable sample contains 21,276 events. An independent CSV counter agrees with the application's totals, valid/absent/invalid-ID counts, distinct accounts, affected accounts, and coverage for every one of the five eligible clients under these settings. Account counts across clients must not be added because a person may submit through more than one client.

Live observations: Screenshot A shows 390 additions for Pano and 198 for Navidrome at 75%. Screenshot B shows 5 and 0 at 1%. Screenshot C verifies Pano's denominator: 520 listens, 360 distinct and affected accounts, 520 absent IDs, and no populated-invalid IDs. Screenshot D shows only Navidrome's numerical evidence in Briefing even though both comparison clients remain selected. The table formats Navidrome's coverage as 52.7%; the precise count-based value is 469/889 = 52.7559%. The screenshots and enlarged evidence crops follow the memo.

4. Reading the result for her decision

Does the number measure what Nora cares about? Partly. The comparison measures the observed gap in submitted IDs and a hypothetical count needed to approach a selected coverage target. Nora needs evidence that a feasible pilot would improve identification. Those are different questions: kind 3, *the real thing or a stand-in*. UUID syntax is a proxy for usable identification, not a check that the ID exists or names the right recording. Missing submission IDs also do not measure missing server matches, which the official JSON documentation describes separately.

There is also a precise mismatch between an approximate rounded count and a minimum-to-reach target. *outreach/filters.py*, function *estimate_scenarios*, calculates *round(min(unusable, max(0, target * total - valid)))*. For Pano at 1%, the unrounded gap is 0.01 * 520 - 0 = 5.2; the function returns 5. Achieved coverage would be 5 / 520 * 100 = 0.961538%. An exhaustive integer search returns 6 as the smallest count reaching at least 1%; 6 / 520 * 100 = 1.153846%. The app follows its documented rounding formula, but that output cannot be read as a guaranteed threshold. The arithmetic is exact for these sample counts; it is not a forecast.

Who is in the rows, and who is not? One source row is a sampled submitted listen, not a unique listener, song, installed client, or maintainer. The app groups accepted rows by playback date, client, version, and internal account key before computing aggregates. The displayed comparison includes named clients that meet the selected account threshold, after classified-importer exclusion and playback-date filtering. Missing client labels and low-reach clients do not appear as eligible comparison candidates. Unknown classifications can remain; importer exclusion is not proof that all historical imports have disappeared. Events without valid accounts can contribute listens but do not contribute distinct-account reach. Users outside this submitted sample and current installations are not counted. This is kind 6, *share of whom*, and kind 5, *which dates belong*: the denominator is selected sample listens, while dates describe playback, not a current active-user census.

What outside the data matters? Nora must establish which component supplies recording IDs, whether metadata can be recovered by mapping, whether a maintainer is active and willing to help, and what effort and benefit a pilot would involve. The public documentation establishes that multiple submission and mapping paths exist; this dataset does not establish feasibility, willingness, costs, or causation. She should check current release and issue information and ask maintainers before treating hypothetical gains as a commitment. No such conversations have been conducted for this case.

What could she wrongly conclude? “Pano has 390 additions at 75%, so contacting its maintainer will create more usable listens than contacting Navidrome” confuses a target-driven scenario with an intervention effect. The clients also start at different coverage levels. “Five fixes achieve 1%” fails the explicit calculation above. Finally, sharing a single-client briefing as if it documented a two-client comparison would omit the competing candidate's numerical evidence. *outreach/views.py*, function *briefing*, filters scenario rows to the inspected client; the comparison-client names in settings do not make those omitted result rows appear.

Reproducible check: From the team repository root, after installing app requirements, run *python scripts/check_case.py*. It prepares the portable sample, independently counts the CSV with Python counters and sets, compares the results with the application, and searches integer additions from zero upward. Output is saved as *cases/tinyu/calculation-evidence.json*. The executed result was *independent_aggregate_check: PASS*; Pano at 1% returned *app_additions: 5*, *minimum_to_reach: 6*, and *achieved_coverage: 0.009615384615384616* (a fraction).

5. The verdict: prove or break

Break the app's readiness to support this recommendation; retain it as a screening aid. Nora is better off using the app to compare observed identifier gaps than grouping raw exports manually, because it makes reach, baseline coverage, and hypothetical changes available together. It is not sufficient evidence to prefer a feasible Pano intervention over Navidrome, nor to promise a 1% milestone. She should return the briefing for correction and investigate submission and mapping paths before choosing an outreach partner.

The following requests are the existing team requests CR-03 and CR-04, originated by Tinyu Xie; they are not new duplicate requests. Their numerical basis has been rechecked here. Both behaviors were confirmed on the public app at the settings above.

CR-03 — Show achieved coverage and the minimum additions needed to reach the target. Part: Scenario comparison. Evidence: Pano, 520 listens, zero valid submitted IDs, 1% target; five rounded additions reach 0.9615%, while six reach at least 1%. Change: preserve the existing approximate scenario number, add its achieved coverage, and separately label the minimum-to-reach count. Priority: medium. Status: open. Acceptance: a regression check distinguishes 5 approximate additions, 0.9615% achieved coverage, and 6 minimum additions; zero and already-met targets return zero. Keep every scenario hypothetical; retain both meanings for all client sizes.

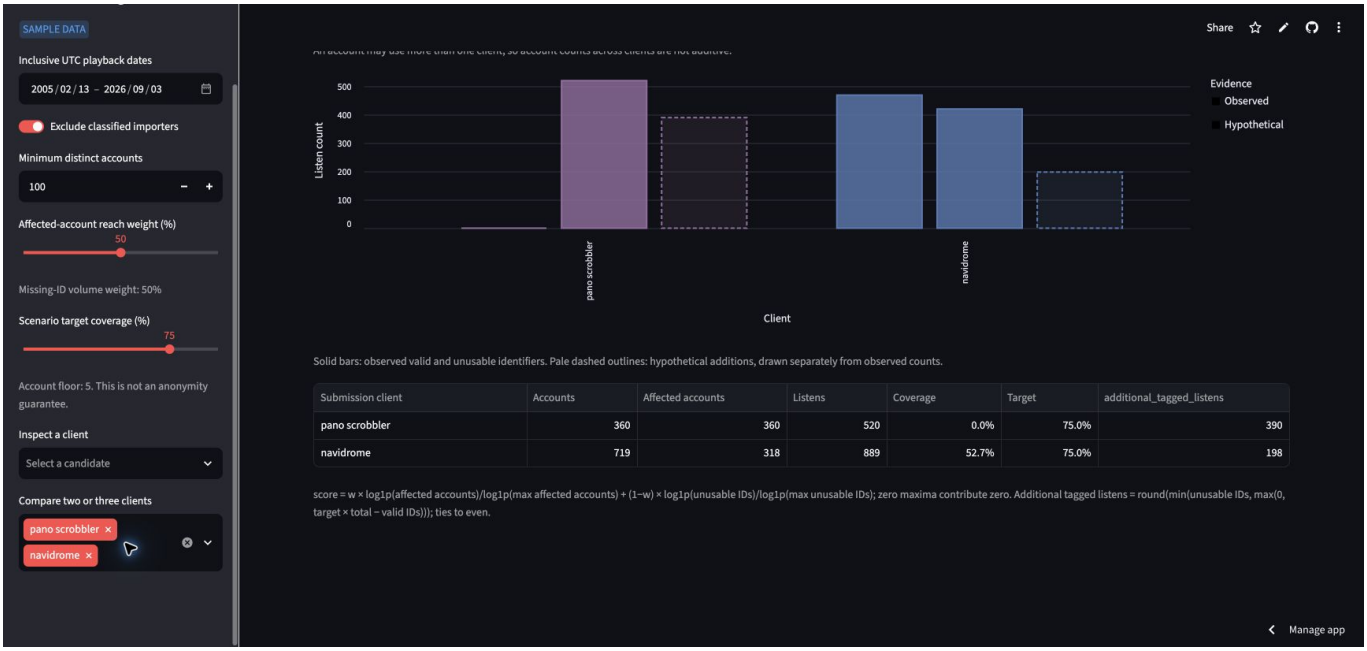
CR-04 — Export the complete selected scenario comparison. Part: Comparison-to-briefing workflow. Evidence: with Navidrome and Pano selected, the reviewed briefing code retains only the inspected client's scenario row. Change: add a complete comparison CSV and Markdown briefing for all selected candidates, including observed totals and coverage, target, additions, source and mode, playback dates, filters, formula, hypothetical-scenario warning, and the warning that account counts are not additive. Preserve the existing individual-client briefing for users needing one candidate. Priority: medium. Status: open. Acceptance: selecting the two clients produces two numerical comparison rows at the same settings and retains reproducibility information.

Appendix — screenshots from the live app

All four screenshots were taken from the public deployment on November 7, 2026. Sample; UTC playback 2005-02-13 to 2026-09-03; importer exclusion on; minimum accounts 100; reach weight 50%; Pano Scrobbler and Navidrome selected.

A. Live scenario comparison — 75% target

Scenario target coverage: 75%. The sidebar shows the selected clients and filters.



Enlarged table crop from the same screenshot. Pano: 390 hypothetical additions. Navidrome: 198.

Submission client	Accounts	Affected accounts	Listens	Coverage	Target	additional_tagged_listens
pano scrobbler	360	360	520	0.0%	75.0%	390
navidrome	719	318	889	52.7%	75.0%	198

B. Live scenario comparison — 1% target

Scenario target coverage: 1%. The sidebar shows the selected clients and filters.

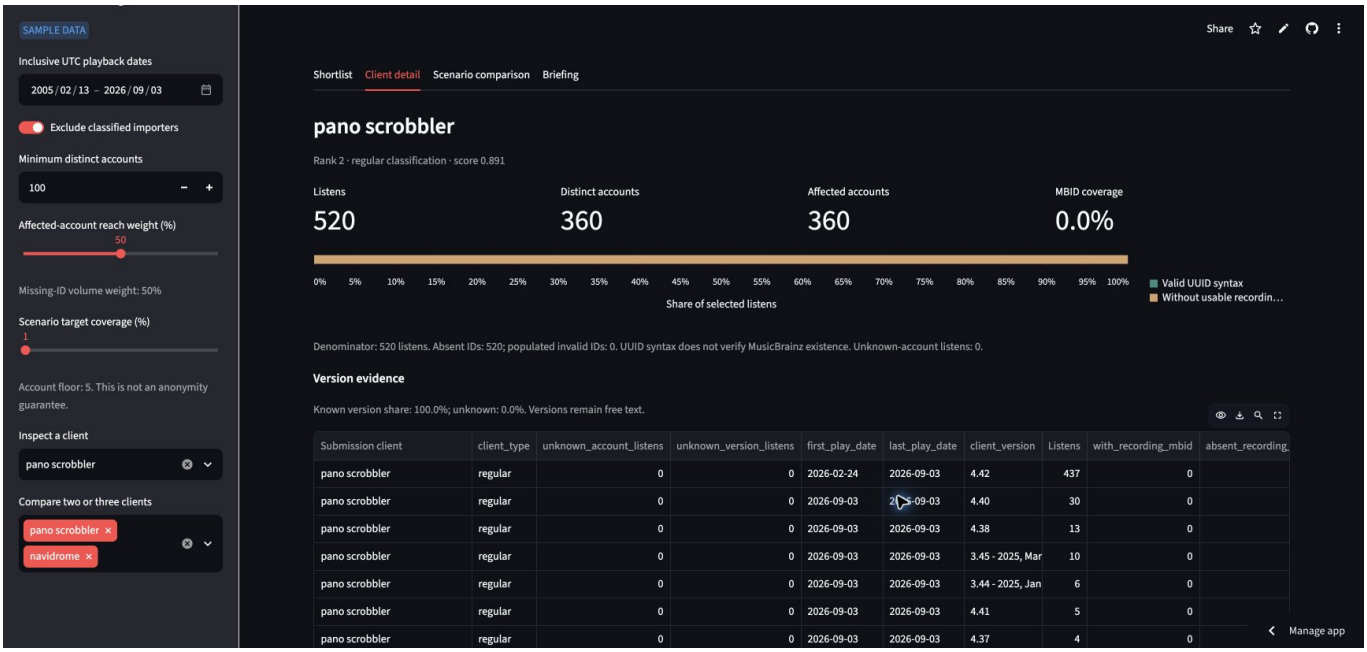


Enlarged table crop from the same screenshot. Pano: 5 approximate additions; Navidrome: 0. Five / 520 = 0.9615%, below the target.

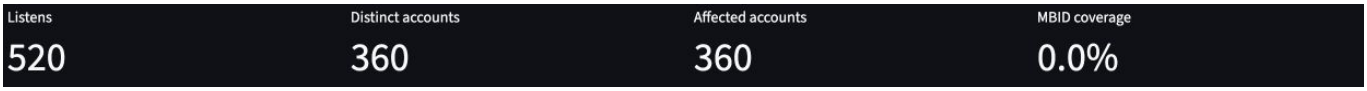
Submission client	Accounts	Affected accounts	Listens	Coverage	Target	additional_tagged_listens
pano scrobbler	360	360	520	0.0%	1.0%	5
navidrome	719	318	889	52.7%	1.0%	0

C. Live Pano detail — the observed denominator

Scenario target: 1%. Pano Scrobbler inspected. The version table continues below the original screenshot.

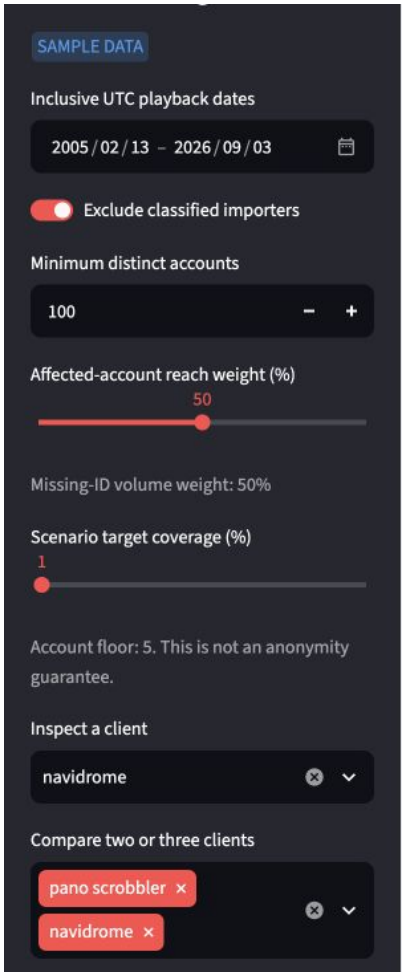


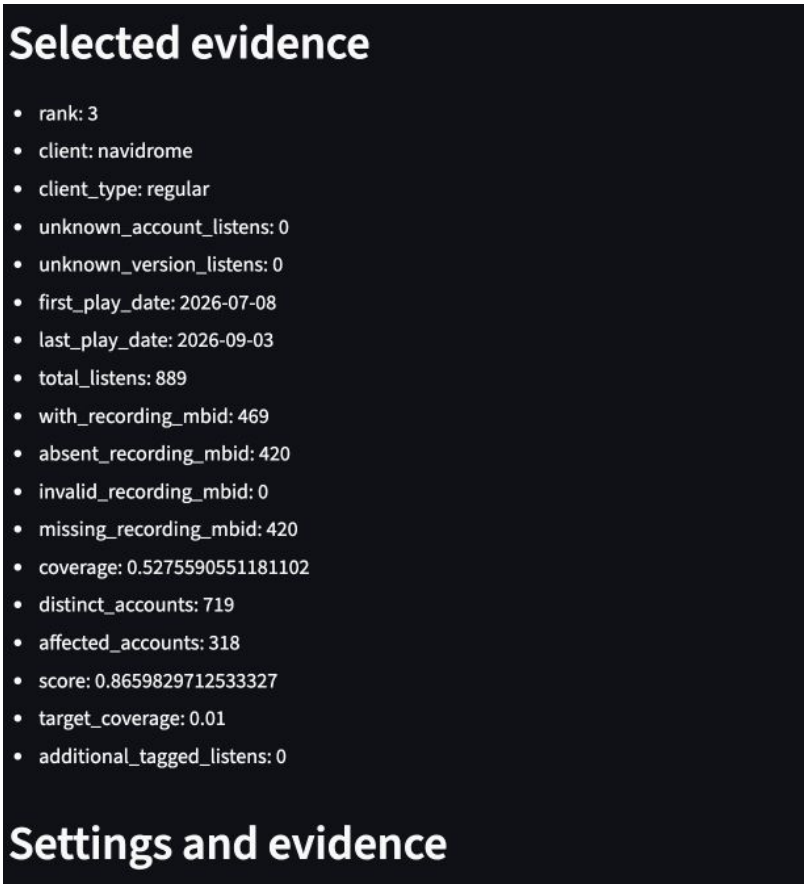
Enlarged observed metrics. The source screen states: denominator 520 listens; 520 absent IDs; 0 populated-invalid IDs; 0 unknown-account listens.



D. Live Briefing — one result with two comparison clients selected

Target: 1%. Navidrome inspected, while Pano Scrobbler and Navidrome both remain selected for comparison. These crops preserve the sidebar settings and the complete Selected evidence block, followed by Settings and evidence.





Should Morgan keep compatibility support for an older Pano Scrobbler version?

Leyna Nguyen | DSO 576 | Individual case | November 8, 2026

Recommendation: Use the Client Metadata Outreach Prioritizer to decide which Pano version to investigate first. Do not use its ListenBrainz counts alone to decide whether to keep or drop compatibility support. The sample does not measure active installations, support requests, reproducible failures, or maintenance cost.

1. The person and the decision

Morgan is a fictional Pano Scrobbler maintainer reviewing limited debugging and compatibility time. Morgan must decide whether version-level ListenBrainz evidence is enough to investigate support needs for an older Pano release before deciding how much support work to keep investing in it. A poor choice could leave an actual problem unresolved or spend effort based on a version count that does not reflect current users or support demand.

2. Morgan's situation and alternatives

Pano's official release notes attach changes and fixes to numbered versions. Its issue tracker also contains a historical version-specific report: a user said AutoShazam detections stopped scrobbling after updating to 3.38 and that 3.37 worked. The maintainer discussed a fix; a later commenter reported the problem on 3.39. That thread shows why reproducing an issue across versions can matter. It does not establish a current compatibility problem or the support needs of older versions. Pano Scrobbler releases, version 3.38 issue.

ListenBrainz defines `submission_client_version` as the version of the software submitting a listen. That makes the field relevant as a possible usage signal. Morgan can also inspect release notes and issues, review support messages and crash logs, reproduce a problem on the affected build and operating system, and ask users whether they can upgrade. Those sources can speak to failure and support burden; the app cannot. ListenBrainz JSON Documentation.

3. The app in use

Live app: Client Metadata Outreach Prioritizer. Morgan would select `pano scrobbler` in client detail, record the playback dates and filters, and review the version table and weekly playback evidence.

Live-app evidence: The screenshots below were captured from the public app on November 8, 2026. They show sample data, playback dates 2005-02-13 through 2026-09-03 UTC, classified-importer exclusion on, a 100-account minimum, 50% affected-account reach weight, and 75% scenario target. The client-detail selection is `pano scrobbler`.

Sample-data evidence: I independently grouped the supplied `sample/listens_20260904.csv` using the app's client-name, playback-timestamp, version-label, and UUID-syntax rules. The sample contains 520 Pano listen events from 360 distinct accounts across 17 reported version labels. All 520 lack a recording MBID; none has a syntactically valid or populated-invalid ID. These are sample calculations, not values verified in a live session.

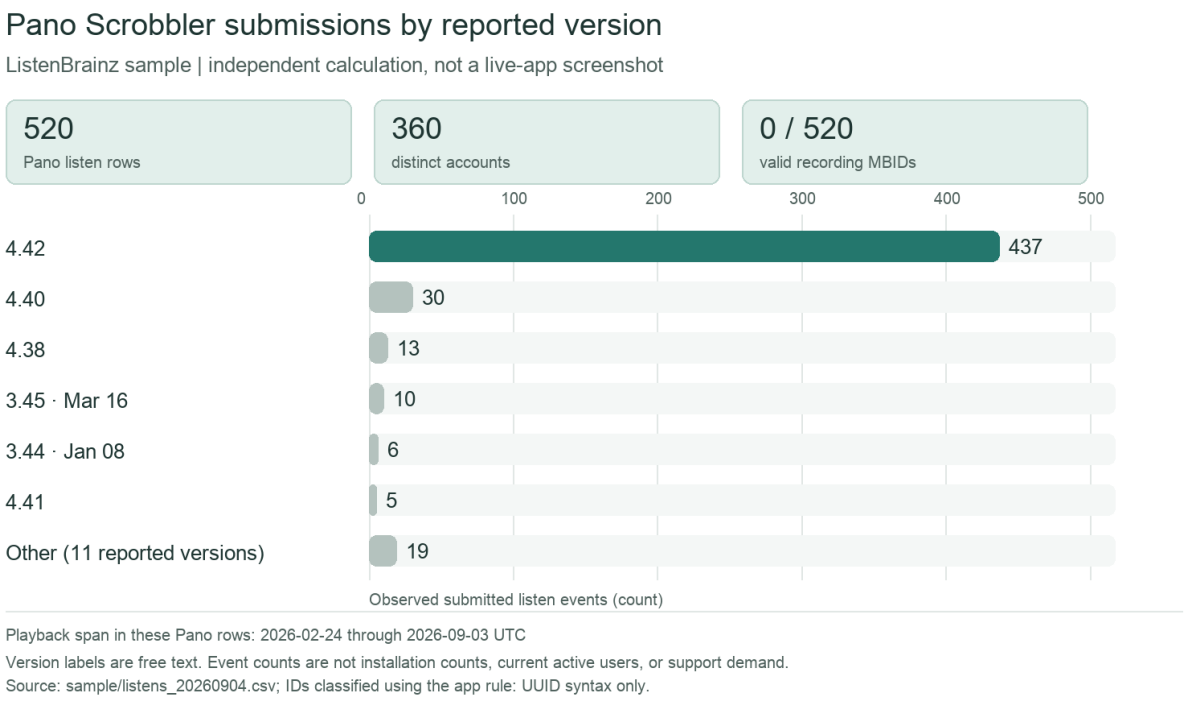


Figure 1. Counts of Pano listen submissions by reported version in the sample; not installation counts or a live-app screenshot.

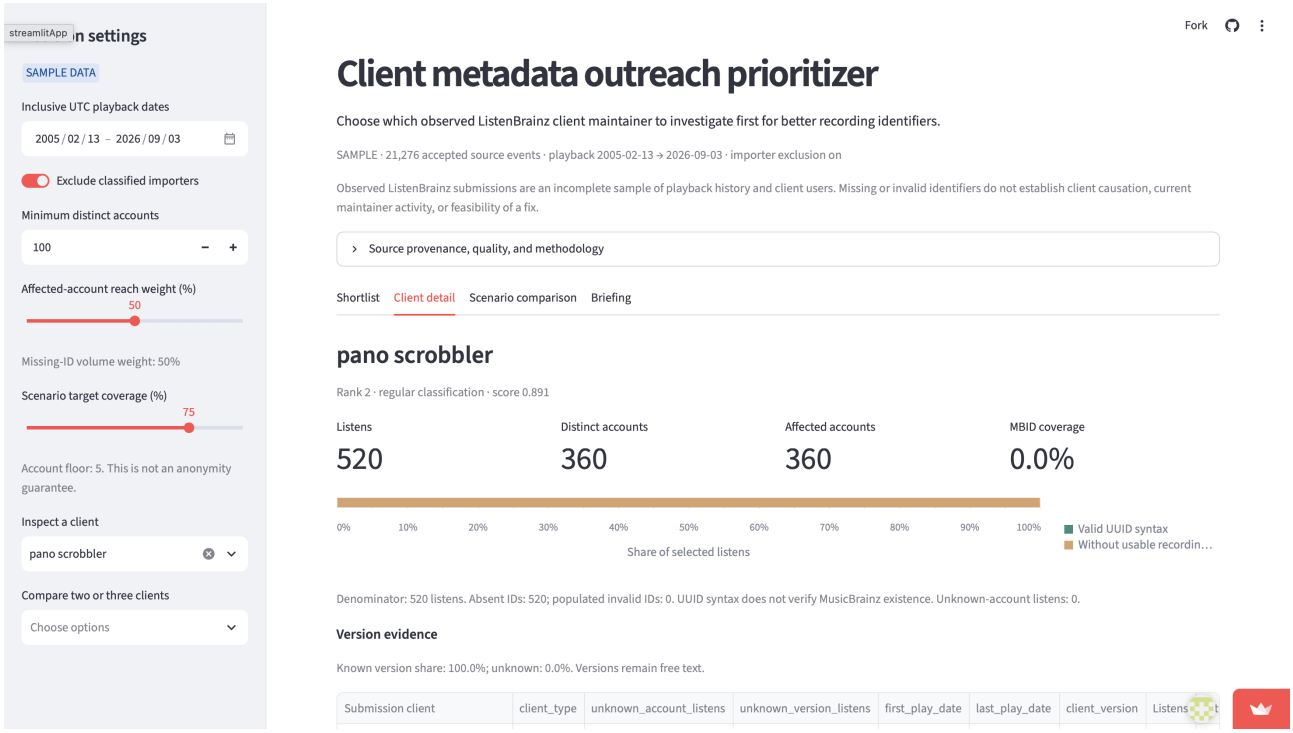


Figure 2. Live client-detail view with Pano Scrobbler selected and summary metrics visible.

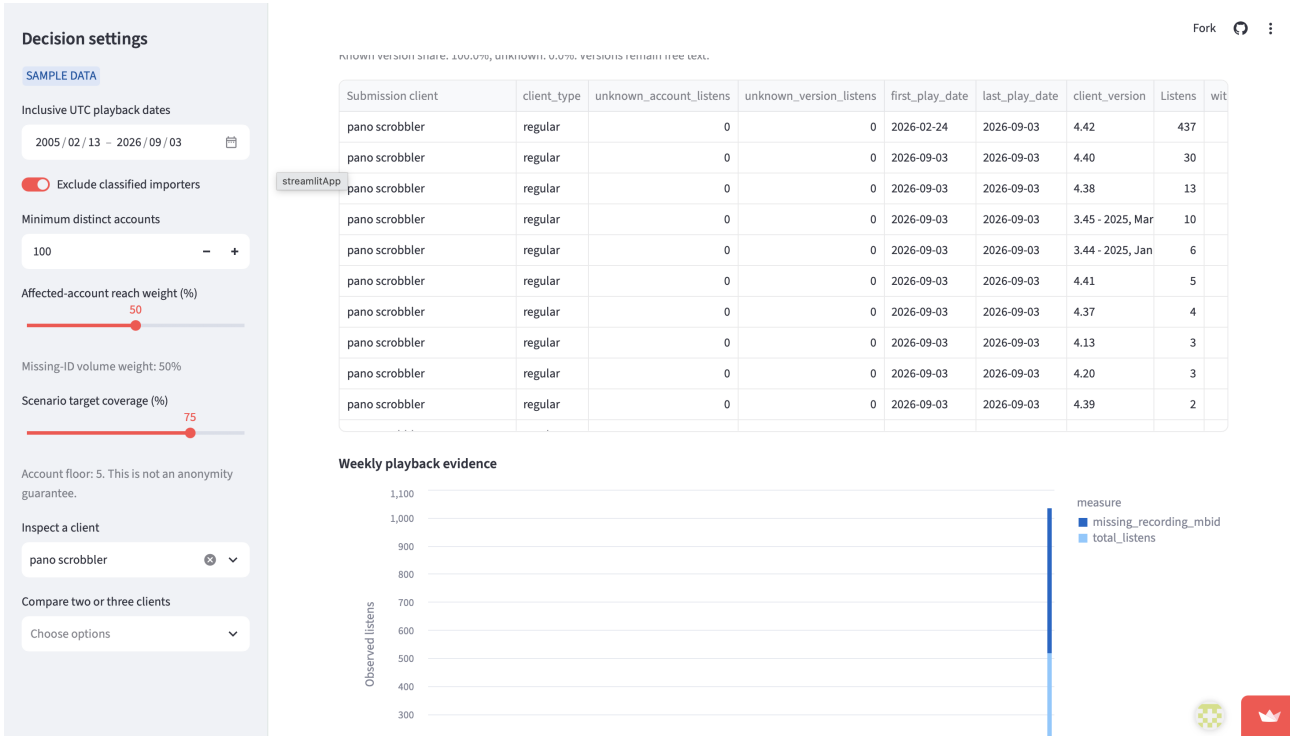


Figure 3. Live version evidence table and weekly playback chart for Pano Scrobbler.

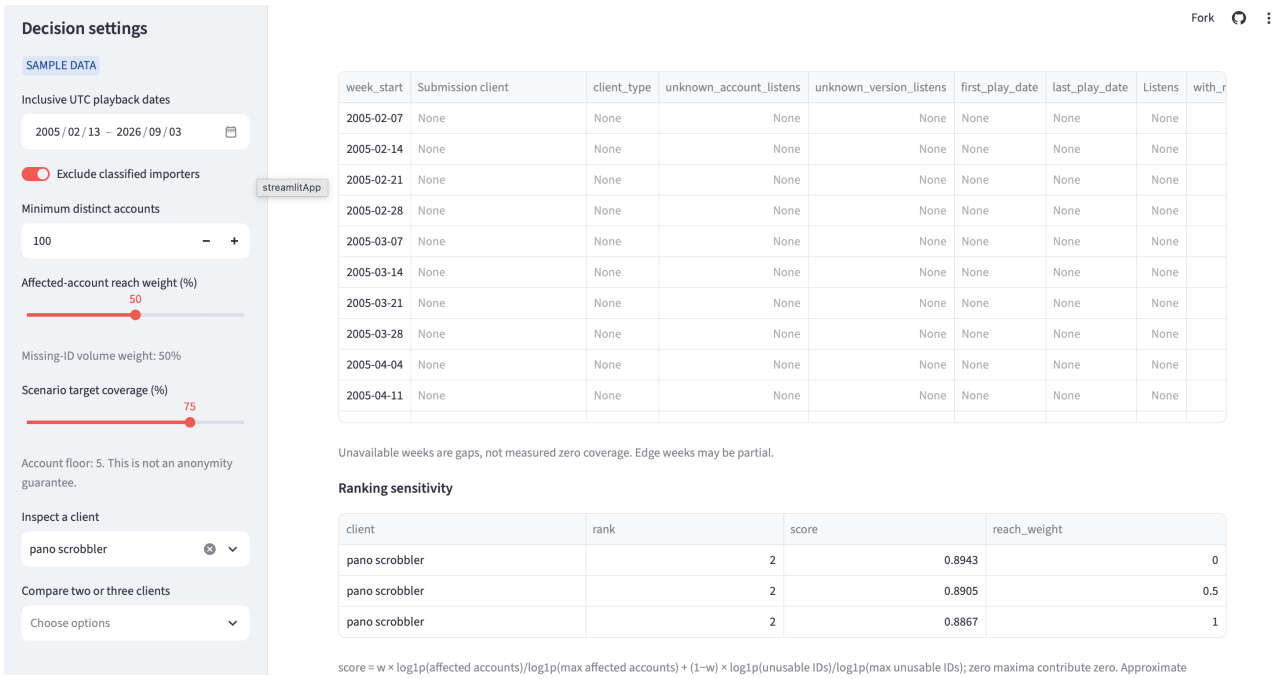


Figure 4. Live weekly evidence table and ranking sensitivity at the displayed reach-weight settings.

The live view reports 520 listens, 360 distinct accounts, 360 affected accounts, 0.0% MBID coverage, rank 2, and score 0.891. Its version table shows 437 listens labeled 4.42 and 30 labeled 4.40. The screenshot values agree with the independent sample calculation above. Version strings are free text, and event counts are not installations or support cases.

4. Reading the result

Does the number measure what Morgan cares about? Only partly. ListenBrainz listens grouped by reported Pano version show submissions in this sample with that version label. The sample has 520 Pano events from 360 accounts; version 4.42 accounts for 437 events. All 520 lack a syntactically valid recording MBID, but that measures submitted identifier data, not whether Pano itself is compatible. The data do not count active installations, support incidents, reproducible failures, engineering effort, or users unable to upgrade. Version traffic can help Morgan choose what to investigate; by itself it cannot decide what to support.

Who is in the rows, and who is not? A source row is one accepted listen submission, not a unique installation, user, bug, or support request. These Pano rows span playback dates 2026-02-24 through 2026-09-03 UTC. Selected dates and importer settings filter rows; events without valid account keys can add listens but not distinct-account counts. The app says observed ListenBrainz submissions are an incomplete sample of playback history and client users. Pano users who do not submit to ListenBrainz, support cases, and listens with unknown or inconsistent version labels are not represented by the version counts.

What outside the data matters? Release notes and issue reports, current support messages and crash logs, operating-system versions, reproducible tests on older builds, upgrade availability, and the maintainer's time. The official issue thread is one historical example of a version-specific report; it is not evidence of current demand.

What could Morgan wrongly conclude? A small observed count does not prove nobody needs an older version; a large count does not prove the version is defective or expensive to support. The 437 events labeled 4.42 are listens, not 437 installations. The sample is incomplete and contains no measure of support workload or compatibility cause. Morgan should review current reports and reproduce a concrete issue before changing the support plan.

5. Verdict

Break for the support-retention decision; keep the app as an investigation aid. The app can narrow which reported Pano version to inspect more efficiently than manually grouping ListenBrainz exports. Morgan should not use its version counts as a census, support-ticket total, or proof that a release is healthy or defective. No new change request is proposed: the missing evidence is support and installation data that this dataset does not contain. Existing team requests CR-05 and CR-06 address version-gap ranking and metric context, but do not measure support demand.

Module 11 Individual Case | Navidrome Version Priority

DSO 576 - Client Metadata Outreach Prioritizer | Observation date: November 8, 2026 | Team deployment, SAMPLE dataset

1. One person, one decision

Person: Alex, a hypothetical ListenBrainz outreach coordinator responsible for selecting an evidence-backed developer conversation. **Decision:** Within the observed Navidrome client, which reported version should Alex investigate first for unusable recording MBIDs before asking a maintainer for help? This decision is distinct from choosing the first client to contact in the entire shortlist.

Alex cares about how many listens could benefit, how many accounts are affected, whether the apparent version gap is real and fixable, and whether a maintainer can act. Without the app, Alex would need to inspect raw submission records, Navidrome documentation and issues, and ask developers about their release history.

2. Live-deployment observations (sample-mode data)

I inspected the team-hosted Streamlit application, not a local development server. Its banner explicitly said SAMPLE, 21,276 accepted source events, playback dates 2005-02-13 through 2026-09-03, with importer exclusion on. Minimum distinct accounts = 100; reach weight = 50%; scenario target = 75%. The observations below concern the app's SAMPLE data, not a census of all Navidrome installations or current production activity.

Shortlist: Navidrome ranked #3, score 0.866, with 889 observed listens, 719 distinct accounts, and 318 affected accounts. **Client detail:** 52.8% recording-MBID coverage. Of 889 listens, 420 lacked usable IDs (420 absent, 0 populated-invalid), leaving 469 with a syntactically usable recording MBID. The reported 52.8% equals 469/889. A valid UUID shape does not prove that an identifier exists in MusicBrainz.

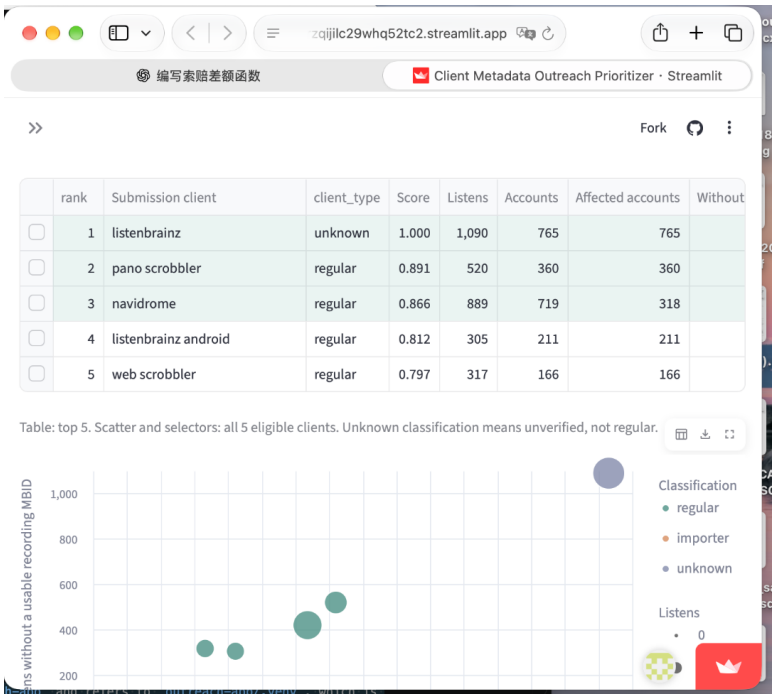


Figure 1. Team-hosted app: Navidrome is third on the shortlist (SAMPLE mode); relevant ranking row and comparison context.

2. Evidence, continued: coverage and version detail

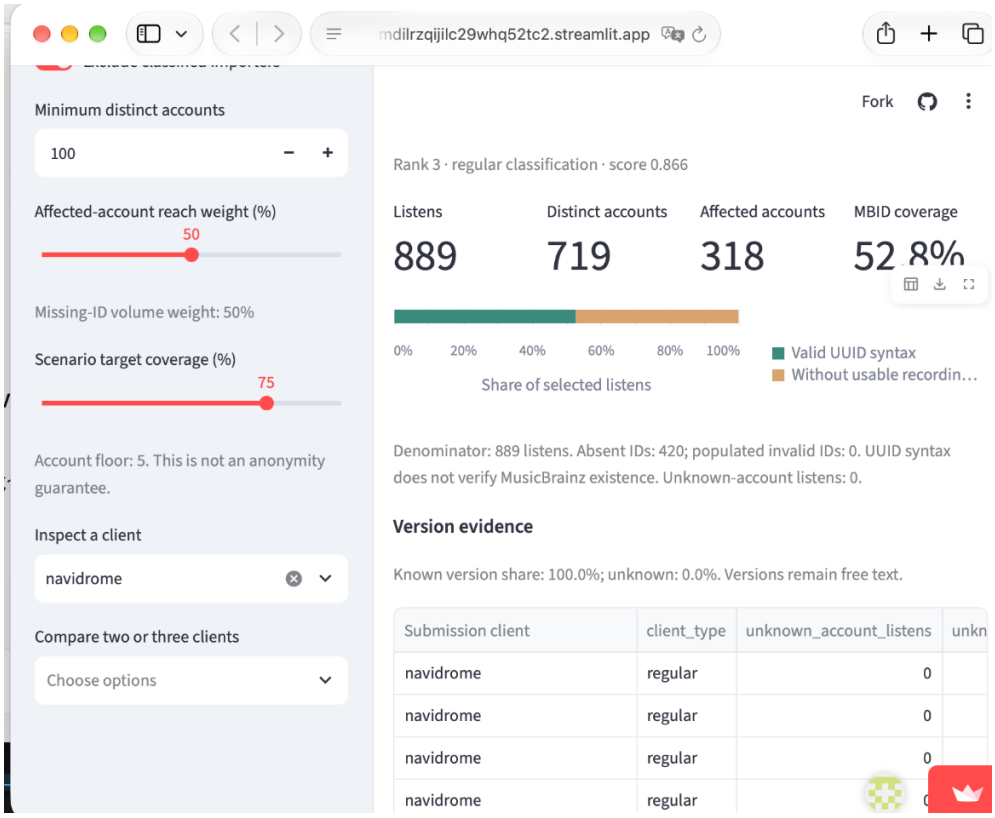


Figure 2. Navidrome client detail: 889 listens, 719 accounts, 318 affected accounts, and 52.8% coverage. Sidebar shows the selected candidate and ranking/scenario controls.

Version evidence: The first visible version row (shown with suffix **be10f89c)**, with its full version string truncated on screen) contains 609 listens, 314 usable IDs and **295 absent IDs** (51.56% coverage). The next visible rows show 39 and 23 absent IDs respectively. This leading version row contributes **295 of Navidrome's 420 unusable-ID listens, or 70.2%**, in this sample. A suffix alone does not establish a verified software release identity.

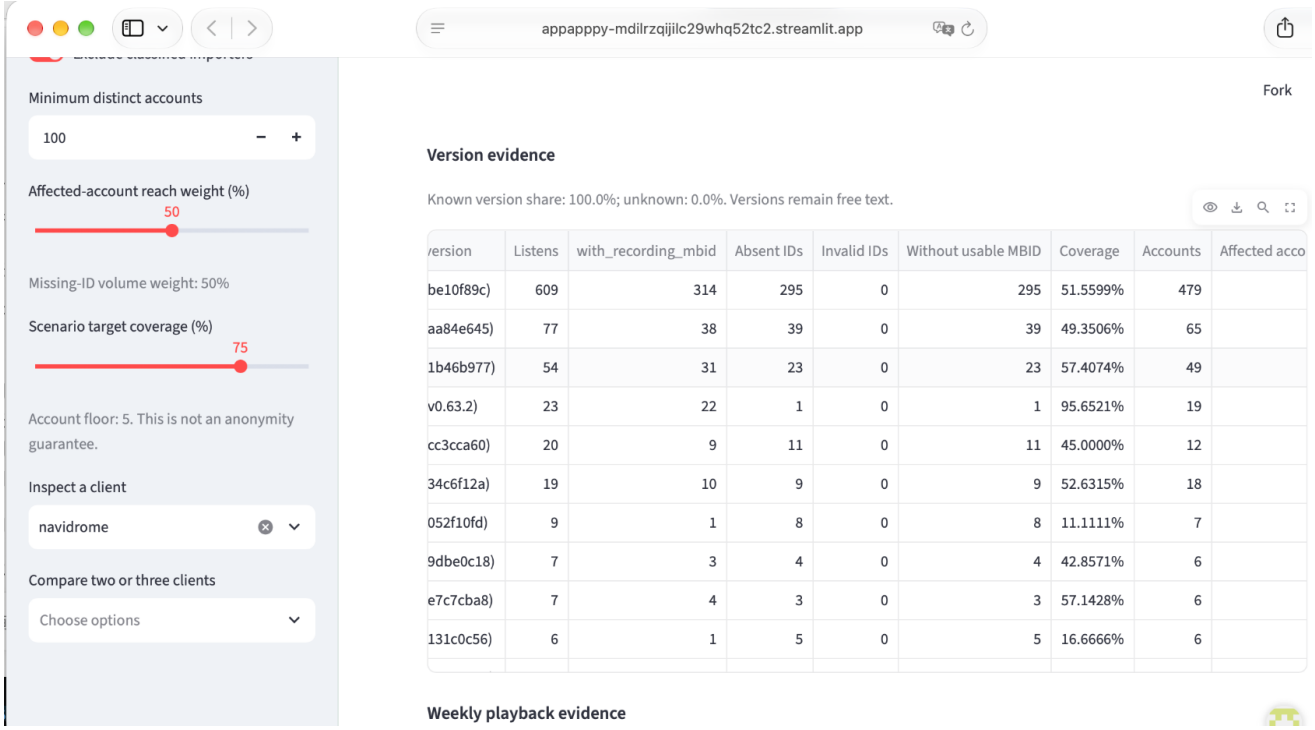


Figure 3. Version evidence from the live deployment. The left edge of version labels is truncated; counts and coverage columns are visible. Do not treat truncated strings as full version identifiers.

3. Does the app measure what Alex needs?

Partly. A missing recording MBID is a useful screening proxy for possible outreach value, and the breakdown makes the first version to investigate visible. But observed misses do not prove that Navidrome caused them, that the maintainer could fix them, or that outreach would recover 295 identifiers. The 52.8% coverage statistic measures syntactic usable-ID presence for filtered submissions, not a verified MusicBrainz match or a success probability. Therefore, the score and the version counts support *investigation*, not automatic outreach.

4. Who is in the rows, and who is not?

Rows represent accepted ListenBrainz source events in the selected sample and playback-date range, after importer exclusion. They are not one row per Navidrome user, installation, software defect, or outreach opportunity. Accounts and listens are different denominators (719 accounts versus 889 listens; 318 accounts affected). Unobserved listens, excluded importers, missing or inconsistent version labels, inactive releases, and users outside the sample are not represented. The 100-account minimum also affects which clients appear in the shortlist. Generalizing the 295 misses to all Navidrome users would create selection bias.

5. What matters outside the data?

Navidrome supports ListenBrainz scrobbling, but an actual fix depends on the client version, library metadata, submission pathway, release history, and whether the maintainers can reproduce the problem. ListenBrainz distinguishes client-submitted identifiers in additional_info from server-resolved mbid_mapping, so a metric must say which kind it counts. Alex should check Navidrome release notes, public issues, and the actual outgoing fields, then ask the maintainer whether the suspected gap is actionable. Neither the sample nor this case establishes a current defect in Navidrome.

6. Verdict - prove or break

PROVE for prioritizing further investigation; BREAK for automatic maintainer outreach. The app improves on manually scanning thousands of records: it narrows attention to Navidrome and the 609-listen version row with 295 missing IDs. For this decision, Alex should investigate that row first, confirm the full version identity, compare against other versions using rates as well as counts, and look for independent technical evidence. Alex should **not** claim the version is defective, promise 295 recoverable MBIDs, rank maintainers by blame, or contact anyone solely from the score.

Relevant team change requests: CR-05 (open) - rank version evidence by missing recording-MBID counts so the highest-gap version is easy to locate; **CR-06 (open)** - show the usable-ID numerator, total listens, calculated coverage, SAMPLE mode, date scope, and limitations beside the metric. These are generalizable improvements, not Navidrome-only fixes. The screenshot shows the version-level counts are available, but not that CR-05 or CR-06 has been implemented as requested.

7. Sources and evidence

1. Team-hosted Streamlit Client Metadata Outreach Prioritizer, observed Nov 8, 2026: Screenshots in Figures 1-3; SAMPLE dataset, filters and values described above.

2. ListenBrainz, JSON Documentation (submission additional_info versus server mbid_mapping): https://listenbrainz.readthedocs.io/en/latest/users/json.html

3. Navidrome, Scrobbling documentation (ListenBrainz integration and per-user settings): https://navidrome.org/docs/usage/scrobbling/

Final submission note: Confirm no teammate has the same person and decision. Share this PDF for team validation.pdf Section 2, and submit your own case PDF in Brightspace.

Ana's diagnostic-review decision: which client should she investigate first?

Gigi Zhu | DSO 576 | Individual case | November 8, 2026

Evidence note. This case uses screenshots supplied by Gigi from the team's public live app on November 7-8, 2026. The all-client rank comparison was reconstructed from the November 8 screenshots. Checks below use displayed sample counts; they are not an independent raw-CSV audit or a new browser-session verification.

Ana should place Pano Scrobbler first in her diagnostic-review queue, while keeping the decision provisional. It is the highest-ranked displayed regular client at all three tested weights, with 360 affected observed accounts and 520 listens missing a recording MBID. The lower-ranked clients do change order, so Pano's stable rank does not establish stability of the entire shortlist. Ana reuses her existing CR-07 to request an all-client ranking comparison instead of relying on separate screenshots.

1. The person and the decision

Ana is a fictional MetaBrainz metadata analyst who has time for one initial client diagnostic review this week. From the five eligible clients, she must decide which client's missing recording-MBID submissions to investigate first. She wants a candidate with substantial observed reach and identifier gaps, and a choice she can explain under different priorities. Choosing poorly would waste her review time on a sample pattern that is not actionable. Her decision sets an investigation queue; it does not approve a pilot or predict the effects of contacting a maintainer.

2. Their real situation and alternatives

MusicBrainz explains that MBIDs identify database entities and distinguish items with ambiguous names [1]. ListenBrainz documents optional recording_mbid, submission_client and client-version metadata, and distinguishes client-submitted IDs from server-resolved mappings [2]. For Ana, missing submitted IDs motivate investigation; they do not establish an API violation, client causation or the absence of a server match.

For this fictional decision, Ana prioritizes observed affected-account reach, missing-ID volume and whether a maintainer could act. Alternatives are to group exports manually in a spreadsheet or Python, inspect submission payloads using the documentation [2], and ask maintainers about the submission path. Manual aggregation can answer count questions, while documentation and maintainer discussion help diagnose feasibility. The app brings the initial screening evidence together, but does not replace these checks. Maintainer capacity, implementation effort and upstream metadata are outside the displayed data; no maintainer conversation was conducted for this case.

Sources and supporting passages, consulted November 7, 2026:

[1] MusicBrainz / MetaBrainz Foundation, MusicBrainz Identifier: MBIDs and disambiguation.
https://musicbrainz.org/doc/MusicBrainz_Identifier

[2] ListenBrainz / MetaBrainz Foundation, JSON Documentation: Submission JSON, Payload JSON details and MBID Mapping.
https://listenbrainz.readthedocs.io/en/latest/users/json.html

3. The app in use

Live app: https://appapppy-mdilrzqijilc29whq52tc2.streamlit.app/
Sample mode; inclusive UTC playback dates 2005-02-13 through 2026-09-03; exclude classified importers; minimum distinct accounts 100. Start at 50% affected-account reach weight and 50% missing-ID volume weight. Leave scenario target coverage at 75%; no scenario outcome is used in this case. Open Shortlist, inspect pano scrobbler in Client detail, and read its version and ranking-sensitivity evidence. Then return to Shortlist and test reach weights 0%, 50% and 100%, holding the other settings constant.

The app reports 21,276 accepted source events, but only five eligible clients and 3,121 listens under the selected filters; 2,652 of those listens lack usable recording MBIDs. At 50% reach weight, listenbrainz ranks first with score 1.000 but has unknown classification. Pano Scrobbler ranks second overall and first among the displayed regular clients. Unknown means unverified, not regular. Ana therefore does not treat the top row as an automatically verified outreach candidate.

Observed sample measures

Pano Scrobbler measure	Observed sample value
Listens / distinct accounts	520 / 360
Affected accounts	360
Absent / populated-invalid recording IDs	520 / 0
Usable submitted-MBID coverage	0.0%
Version 4.42 listens	437 (84.04% of 520)
Scores at 0% / 50% / 100% reach	0.894 / 0.891 / 0.887

All-client ranks at the three reach weights

Submission client	0% reach	50% reach	100% reach
listenbrainz	1	1	1
pano scrobbler	2	2	2
navidrome	3	3	3
listenbrainz android	5	4	4
web scrobbler	4	5	5

The November 8 screenshots reproduce the earlier CR-07 rank change: Web Scrobbler is fourth at 0% reach weight, while ListenBrainz Android is fourth at 50% and 100%. Web Scrobbler has more unusable-ID listens (317 versus 305), but Android has more affected accounts (211 versus 166). At 0% reach weight, only missing-ID volume receives weight; increasing reach weight favors Android enough to reverse their order. Pano stays second at all three settings. This is a three-setting check on the selected sample, not a proof for every intermediate weight or future dataset.

Displayed-count checks: Pano's usable listens = 520 - 520 absent - 0 invalid = 0; listen coverage = 0 / 520 = 0.00%. Affected share of observed Pano accounts = 360 / 360 = 100.00%. Version 4.42 contributes 437 / 520 = 84.04% of Pano's observed listens. These are arithmetic checks on screenshot values, not tests of the underlying CSV or estimates for the full user population.

4. Reading the result for their decision

Does the number measure what this person cares about? Partly. The 360 affected observed accounts and 520 absent-ID listens address Ana's initial screening priorities. Coverage is a listen-level measure, however, and usable here means valid UUID syntax; the detail page says syntax does not verify MusicBrainz existence. This is kind 3, the real thing or a stand-in: syntactic coverage is not confirmed recording identity, maintainer willingness or achievable benefit. The ranking score is not a success probability.

Who is in the rows, and who is not? The source rows are observed listening events, not unique songs, installed clients or all users. The shortlist excludes classified importers and clients below the selected account threshold, but unknown classifications can remain. This is kind 6, share of whom: Pano's 0/520 describes selected observed listens, not every listen through that client. It is also kind 5, which dates belong: these are playback dates, not a census of current active installations. Version 4.42 dominates the sample; weekly gaps are unavailable observations, not measured zero coverage.

4. Reading the result (continued)

What outside the data matters to this decision? Ana must establish which component supplies IDs, whether upstream metadata is available, and whether a maintainer is active and able to help. Source [2] permits omission of unavailable optional metadata, so missing IDs are not automatically evidence of broken software. Source [1] concerns identifiers of particular entities, so a well-formed UUID alone is insufficient to confirm the correct recording. The app does not measure implementation effort or the causal impact of outreach.

What could they wrongly conclude from it? Pano's stable rank 2 could be misread as proof that all candidate priorities are stable. The all-client comparison disproves that: Android and Web Scrobbler exchange fourth and fifth place. Ana could also mistake the 0% sample coverage for a statement about all Pano users, or assume that contacting the maintainer will fix all 520 missing IDs. Neither follows. The appropriate conclusion is that Pano is a reasonable first investigation target under these settings, while lower-priority choices depend on the user's reach-versus-volume preferences.

5. The verdict: prove or break

Break the completeness of the ranking-comparison workflow, while retaining the app as a screening aid. Ana provisionally puts Pano Scrobbler first in her diagnostic-review queue: it combines 360 affected observed accounts, 520 absent-ID listens, regular classification and a stable rank across the three tested weights. She is better off with the app's shortlist and detail evidence than sorting raw listen totals alone, but should examine payloads for the observed versions and investigate feasibility before making a separate outreach decision. She must not use the app to assign blame, describe all client users, approve a pilot or promise a coverage target.

The following is Gigi Zhu's existing CR-07 from the previous assignment, not a new duplicate request. Its original date, authorship and historical evidence are retained. The supplied team log lists its status as open.

CR-07: Compare Rankings Across Different Weights

From: Gigi Zhu, 2026-10-26
Part: Shortlist ranking feature (Part 1)
Evidence: In Sample Data mode, with playback dates 2005-02-13 through 2026-09-03, classified importers excluded and minimum accounts set to 100, I compared reach weights of 0%, 50% and 100%. Web scrobbler ranked fourth at 0%, while listenbrainz android ranked fourth at 50% and 100%. The Shortlist table shows only the current setting, so comparing results required screenshots.
Change: Add a side-by-side comparison of all shortlisted clients' ranks at 0%, 50% and 100% reach weight. Explain rank changes using affected-account counts and unusable-ID listen counts so users can understand how their priorities affect the recommendation.
Priority: medium
Status: open

Case follow-up, November 8, 2026. Current live-app screenshots confirm all five ranks at 0%, 50% and 100%, including the Android/Web Scrobbler reversal. The current Client detail screenshot supplies a sensitivity table for Pano alone. Reconstructing the all-client comparison still required the separate Shortlist screenshots used here. These captures do not establish that no equivalent view exists elsewhere in the deployed app; verify that before marking CR-07 built. The requested improvement remains a consolidated all-client rank table with the same date/filter settings and an explanation tied to reach and unusable-ID counts.

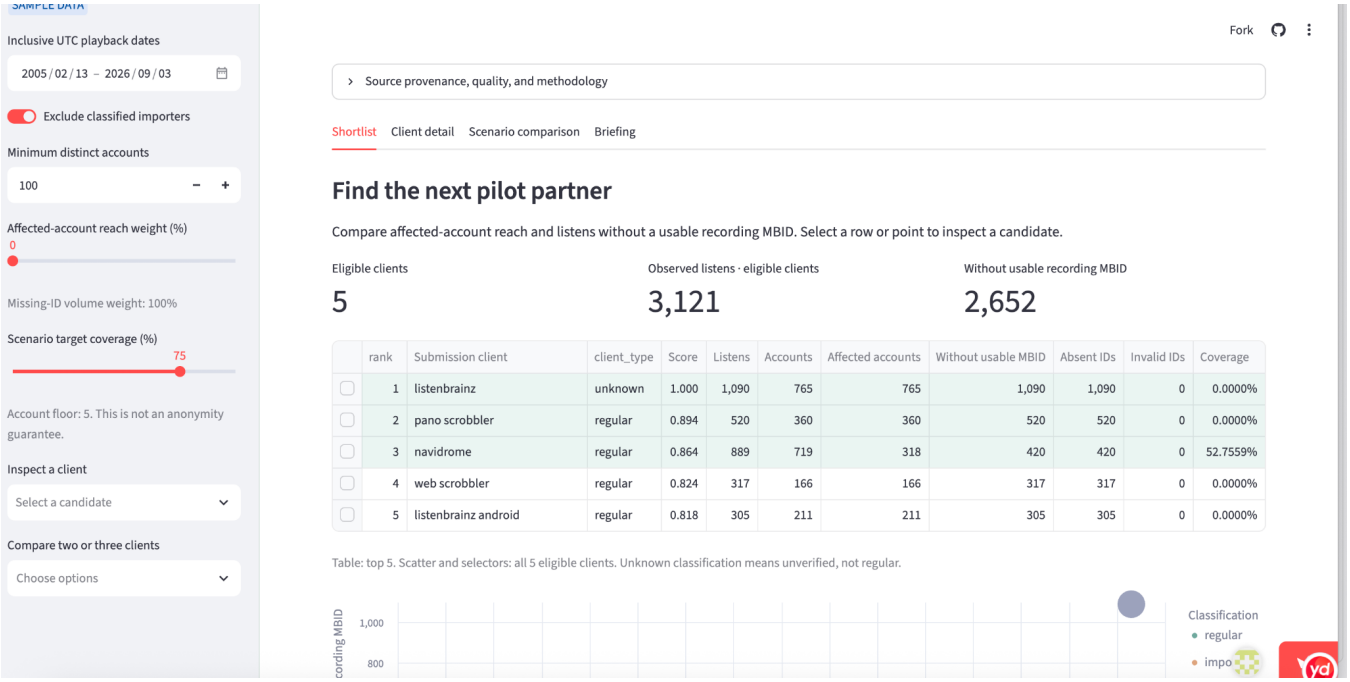
Acceptance and impact. At these sample settings, the comparison should reproduce the five-client table in this case and explain why 317 versus 305 unusable-ID listens favors Web Scrobbler at 0%, while 211 versus 166 affected accounts favors Android as reach weight increases. This is a proposed acceptance check for the consolidated view, not an automated test executed here. Apply the view to every eligible client and retain the existing counts, ranking formula and workflows; other users and teammates' cases keep the same numerical inputs. Update the existing CR-07 history instead of adding a new number.

Appendix - screenshots from the live app

All screenshots below were supplied by Gigi. November 8 Shortlist evidence uses sample mode, playback 2005-02-13 through 2026-09-03, importer exclusion on, minimum accounts 100 and scenario target 75%. November 7 detail evidence uses the same baseline settings.

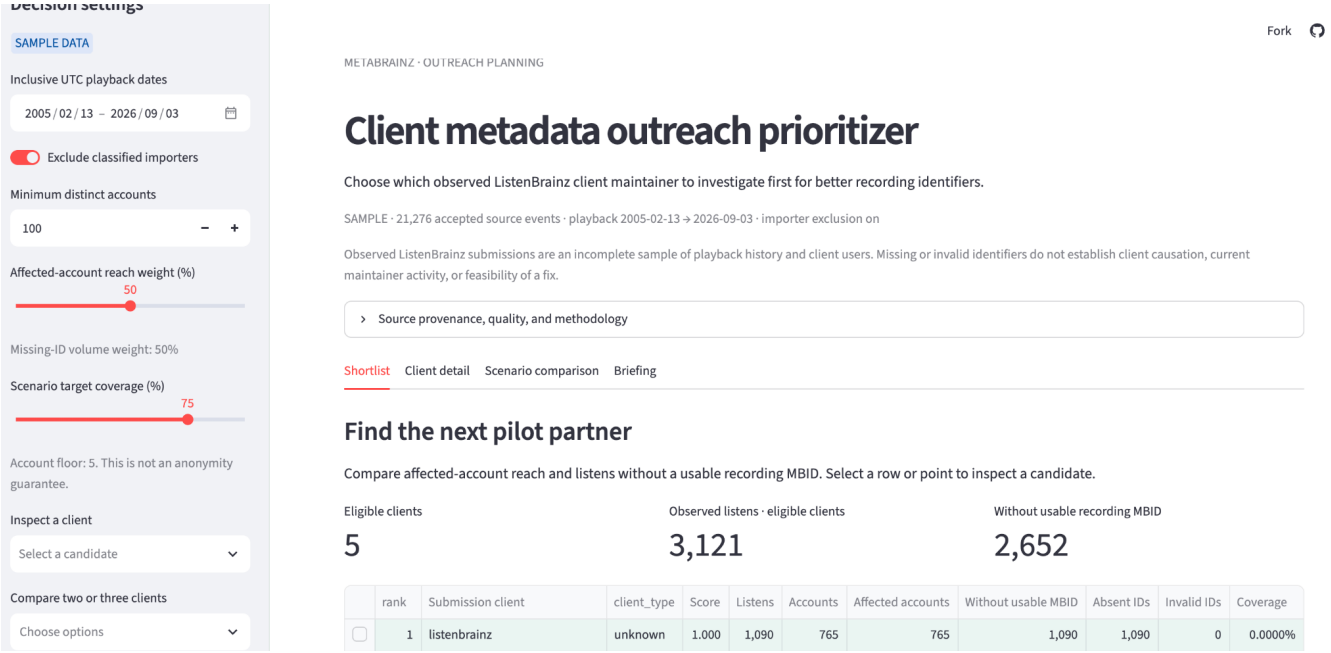
A. Live Shortlist - 0% reach weight

Reach 0%; missing-ID volume 100%; scenario target 75%. All five ranks and shared filters are visible.



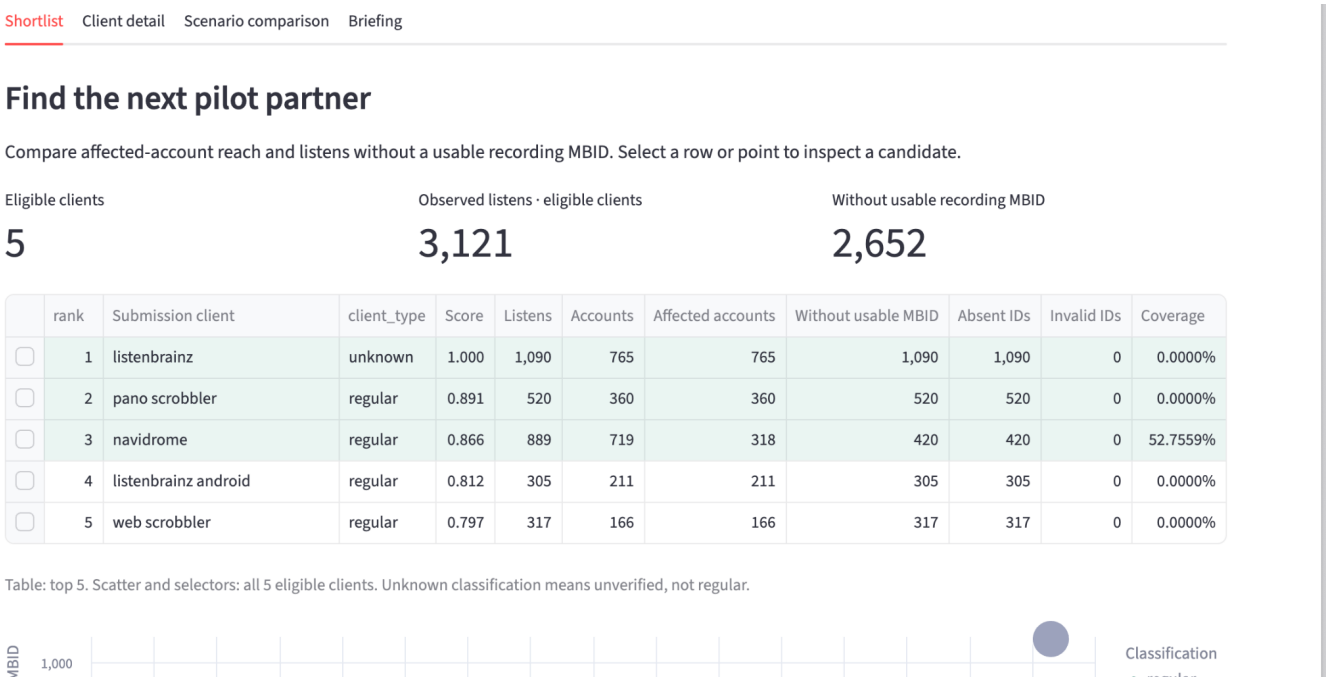
B. Live Shortlist - 50% reach weight

Reach 50%; missing-ID volume 50%; scenario target 75%. Settings and table were captured in consecutive screenshots with the same setting.



B. Live Shortlist - complete 50% table

The full table complements the preceding settings screenshot.



Shortlist | Client detail | Scenario comparison | Briefing

Find the next pilot partner

Compare affected-account reach and listens without a usable recording MBID. Select a row or point to inspect a candidate.

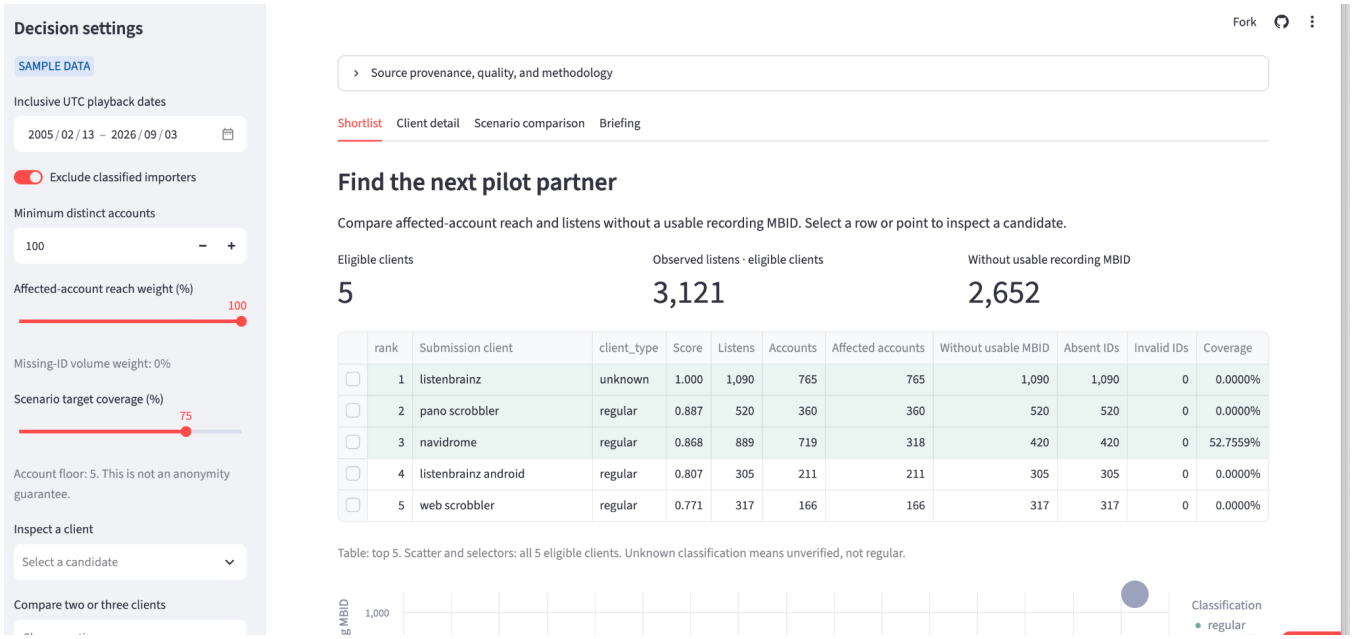
Eligible clients	Observed listens · eligible clients	Without usable recording MBID
5	3,121	2,652

rank	Submission client	client_type	Score	Listens	Accounts	Affected accounts	Without usable MBID	Absent IDs	Invalid IDs	Coverage
1	listenbrainz	unknown	1.000	1,090	765	765	1,090	1,090	0	0.0000%
2	pano scrobbler	regular	0.891	520	360	360	520	520	0	0.0000%
3	navidrome	regular	0.866	889	719	318	420	420	0	52.7559%
4	listenbrainz android	regular	0.812	305	211	211	305	305	0	0.0000%
5	web scrobbler	regular	0.797	317	166	166	317	317	0	0.0000%

Table: top 5. Scatter and selectors: all 5 eligible clients. Unknown classification means unverified, not regular.

C. Live Shortlist - 100% reach weight

Reach 100%; missing-ID volume 0%; scenario target 75%. Android ranks fourth and Web Scrobbler fifth.



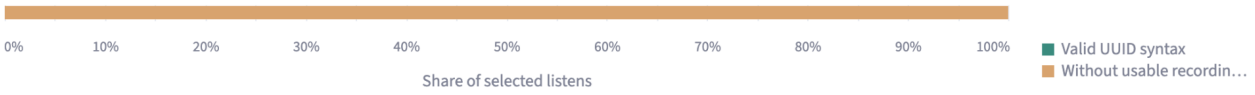
D. Live Pano Client detail

520 listens, 360 distinct and affected accounts, 520 absent IDs and 0 populated-invalid IDs. Version evidence is partially visible; no claim relies on cut-off columns.

pano scrobbler

Rank 2 · regular classification · score 0.891

Listens	Distinct accounts	Affected accounts	MBID coverage
520	360	360	0.0%



Denominator: 520 listens. Absent IDs: 520; populated invalid IDs: 0. UUID syntax does not verify MusicBrainz existence. Unknown-account listens: 0.

Version evidence

Known version share: 100.0%; unknown: 0.0%. Versions remain free text.

Submission client	client_type	unknown_account_listens	unknown_version_listens	first_play_date	last_play_date	client_version	Listens	with_re
pano scrobbler	regular	0	0	2026-02-24	2026-09-03	4.42	437	
pano scrobbler	regular	0	0	2026-09-03	2026-09-03	4.40	30	
pano scrobbler	regular	0	0	2026-09-03	2026-09-03	4.38	13	
pano scrobbler	regular	0	0	2026-09-03	2026-09-03	3.45 - 2025, Mar	10	
pano scrobbler	regular	0	0	2026-09-03	2026-09-03	3.44 - 2025, Jan	6	

E. Live Pano ranking sensitivity

Pano remains at rank 2 at reach weights 0, 0.5 and 1. This is one-client evidence.

Unavailable weeks are gaps, not measured zero coverage. Edge weeks may be partial.

Ranking sensitivity

client	rank	score	reach_weight
pano scrobbler	2	0.8943	0
pano scrobbler	2	0.8905	0.5
pano scrobbler	2	0.8867	1

score = w × log1p(affected accounts)/log1p(max affected accounts) + (1−w) × log1p(unusable IDs)/log1p(max unusable IDs); zero maxima contribute zero. Approximate additions = round(min(unusable IDs, max(0, target × total − valid IDs))); ties to even. Achieved coverage = (valid IDs + approximate additions) / total. Minimum additions = max(0, ceil(target × total − valid IDs)), capped at unusable IDs; decimal target arithmetic.

F. Live weekly playback evidence

Sparse observed playback weeks limit temporal generalization. No observations in a week does not mean a measured zero rate.



Rachel and the listenbrainz outreach decision

> Bottom line: Rachel should use the app to investigate listenbrainz, but should hold it for classification and ownership verification before adding it to the outreach candidate list. The app's rank is a useful screening signal, not proof that the label identifies a contactable client maintainer or an eligible outreach target.

1. The person and the decision

Rachel is a fictional ListenBrainz integration coordinator who maintains a list of submission clients that the MetaBrainz Foundation may investigate and contact. When a client appears at the top of the shortlist, Rachel must decide whether to add it to the outreach candidate list now or hold it for classification verification first. A poor decision could send outreach toward an unverified label or cause Rachel to overlook a real client that needs better recording metadata.

Her case is the client label listenbrainz, which ranks first with score 1.000 but is marked unknown by the app.

2. Rachel's situation and alternatives

ListenBrainz's official JSON documentation (https://listenbrainz.readthedocs.io/en/latest/users/json.html) defines submission_client as the client used to submit a listen and submission_client_version as that client's version. It also defines original_submission_client as the first client when another tool later imports a listen and notes that this field is useful for importers. These fields make the observed label useful for investigation, but they do not establish an owner, maintainer, or outreach eligibility.

The official ListenBrainz Core API documentation (https://listenbrainz.readthedocs.io/en/latest/users/api/core.html) describes submission through submit-listens and distinguishes ordinary submissions from import submissions. Rachel should therefore verify the submission context and metadata before interpreting an aggregate label as a direct-client population.

Rachel's alternatives are to inspect the exact label in the ListenBrainz/MetaBrainz code and issue tracker, search for a repository owner or client documentation, ask the ListenBrainz community, and use current support or maintainer evidence. The MetaBrainz ListenBrainz server repository (https://github.com/metabrainz/listenbrainz-server) directs contributors to its issue tracker and IRC/Matrix/Discord community, giving Rachel public routes for verification and contact.

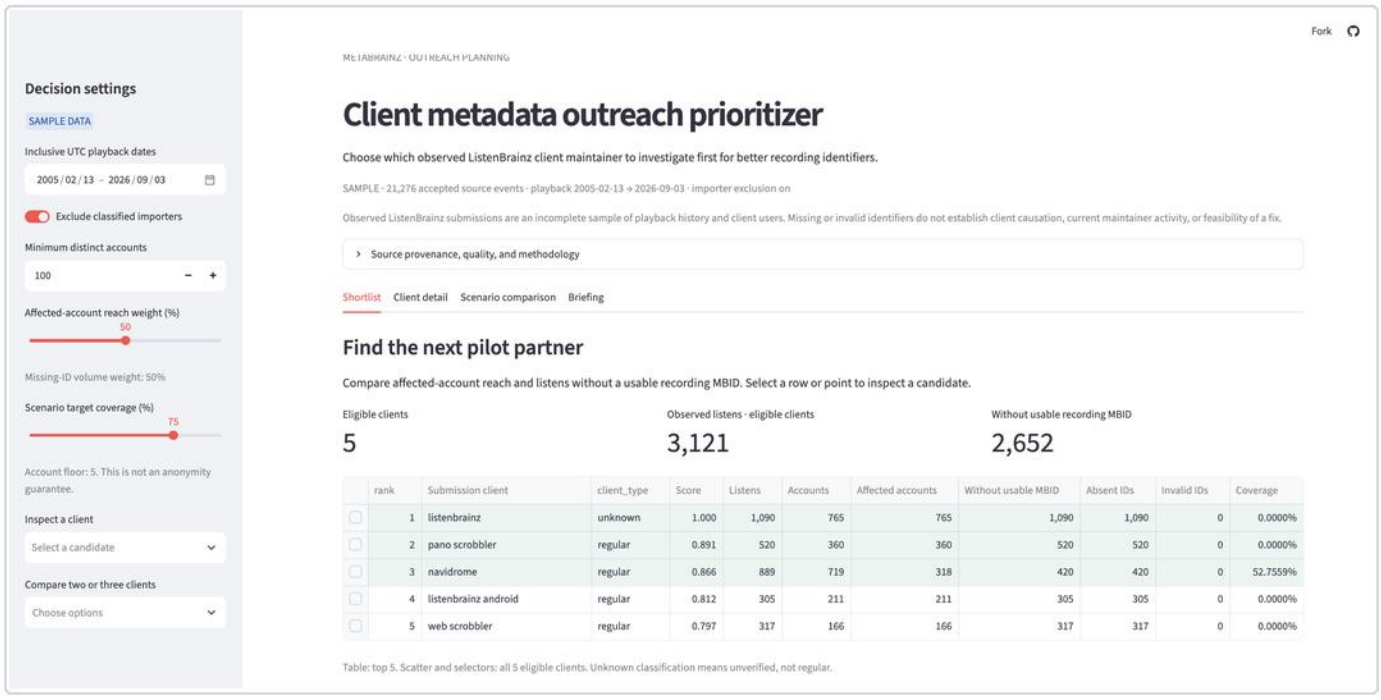
The app does not contain a verified ownership directory, an importer/first-party provenance field, current maintainer activity, support demand, or a causal test showing that the client caused missing recording MBIDs. Those facts matter to Rachel's decision but must come from outside the sample.

3. The app in use

Rachel uses the Client Metadata Outreach Prioritizer (https://appapppy-mdilrzqijilc29whq52tc2.streamlit.app/) in Sample Data mode. The relevant settings are:

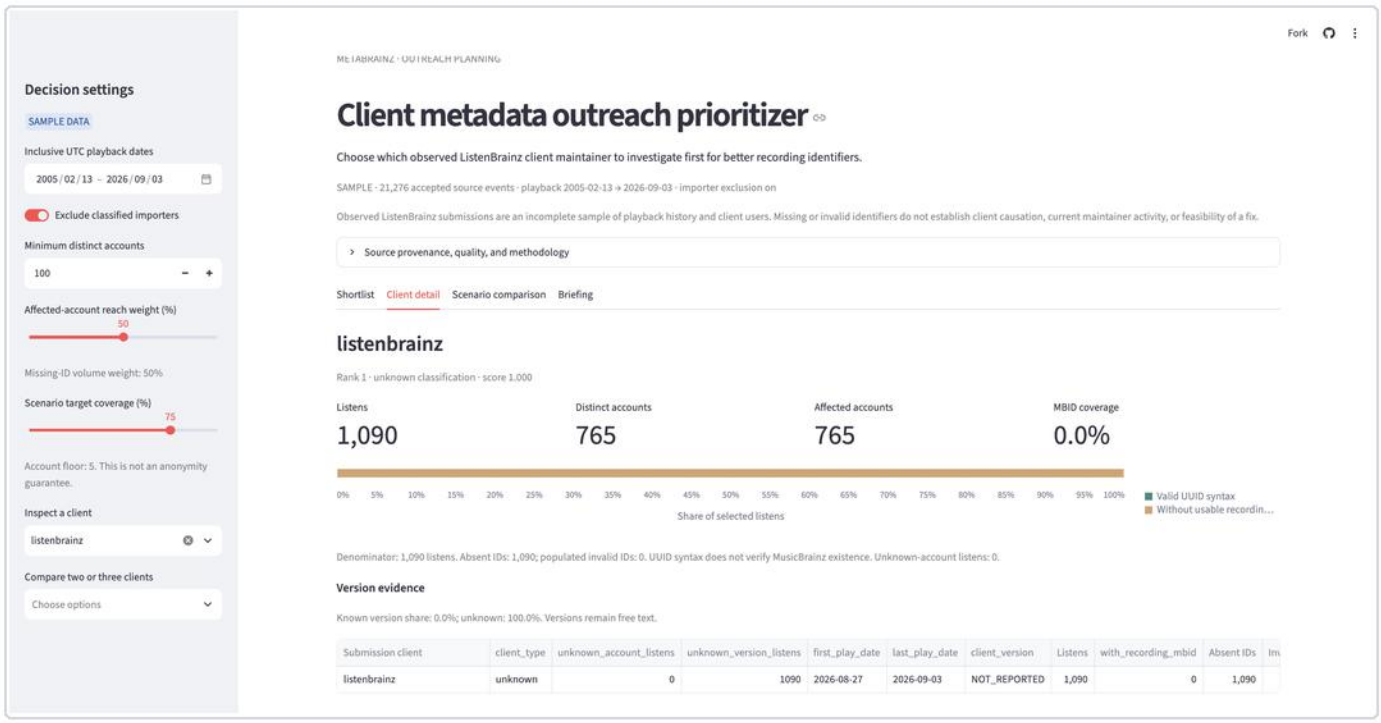
- playback dates: 2005-02-13 through 2026-09-03 UTC;
- classified-importer exclusion: on;
- minimum distinct accounts: 100;
- affected-account reach weight: 50%;
- scenario target: 75%.

She inspects listenbrainz on the Shortlist and then selects it in Client detail.

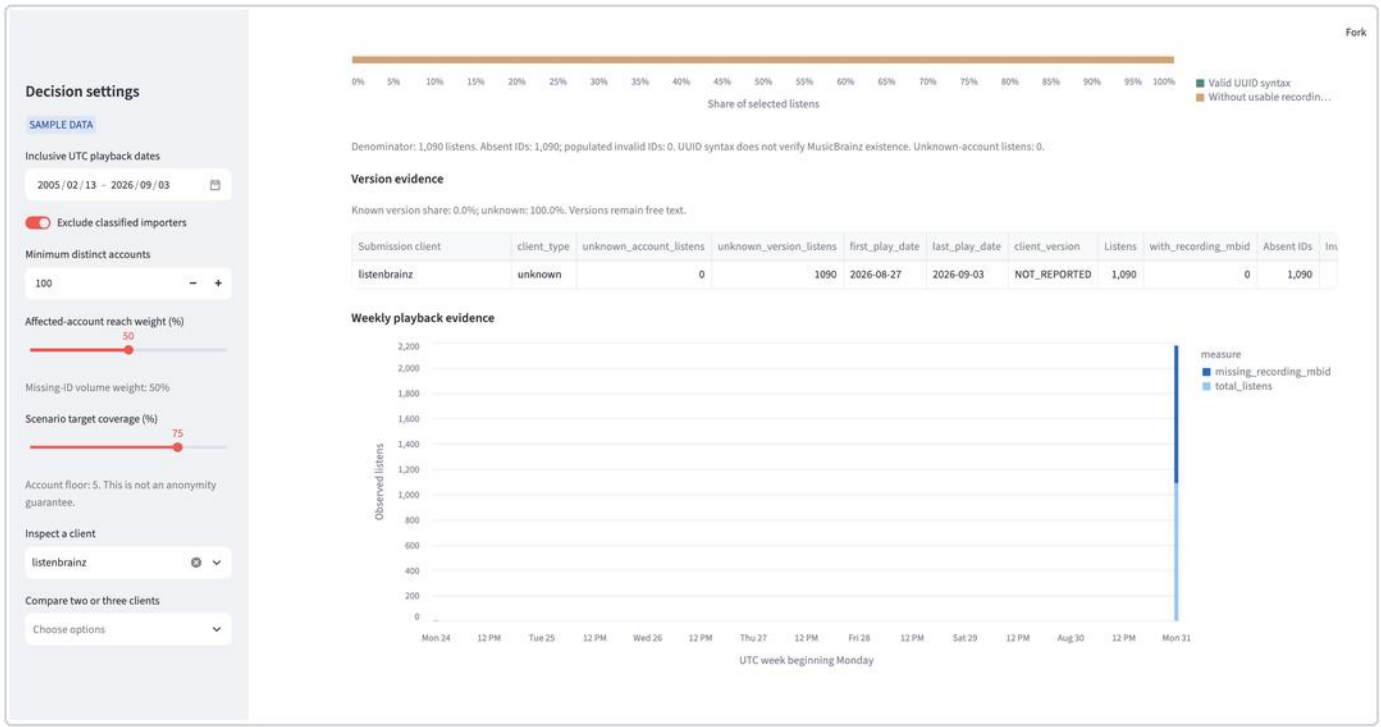


rank	Submission client	client_type	Score	Listens	Accounts	Affected accounts	Without usable MBID	Absent IDs	Invalid IDs	Coverage
1	listenbrainz	unknown	1.000	1,090	765	765	1,090	1,090	0	0.0000%
2	pano scrobbler	regular	0.891	520	360	360	520	520	0	0.0000%
3	navidrome	regular	0.866	889	719	318	420	420	0	52.7559%
4	listenbrainz android	regular	0.812	305	211	211	305	305	0	0.0000%
5	web scrobbler	regular	0.797	317	166	166	317	317	0	0.0000%

Live Shortlist with Rachel's settings and listenbrainz ranked first.



Live Client detail for listenbrainz, including its unknown classification and score.



Live evidence or provenance view showing the sample mode and playback scope.

The live screenshots show 1,090 observed listens, 765 distinct accounts, 765 affected accounts, 1,090 unusable recording MBIDs, 0 valid recording MBIDs, and 0.0% coverage. They also show rank 1, score 1.000, and classification unknown. The evidence view shows that all 1,090 listens have an unreported version and that the selected client's observed playback dates are 2026-08-27 through 2026-09-03 within the broader sample range.

4. Reading the result for Rachel's decision

Does the number measure what Rachel cares about?

Only partly. The score measures the size of the observed listenbrainz aggregate relative to the other rows on two screening inputs: affected accounts and listens without a usable recording MBID. It does not measure whether listenbrainz is a standalone submission client, who maintains it, whether it is an importer, or whether anyone is available to contact.

The code confirms this distinction. rank_clients computes the score from affected_accounts and missing_recording_mbid. Separately, clean_event assigns a client type from exact configured importer and regular-client lists. Ranking and outreach eligibility are therefore not the same decision.

Who is in the rows, and who is not?

Each row represents one accepted ListenBrainz submission event. In the sample, the listenbrainz aggregate represents 1,090 events from 765 observed accounts across the selected playback dates—not 765 installations, maintainers, organizations, or outreach requests. Rows without valid positive account identifiers can contribute listens but not distinct-account counts.

The data leave out people who did not submit to ListenBrainz, support reports, inactive installations, ownership information, and a reliable distinction between a direct client and an importer unless metadata and classification rules provide it.

What outside the data matters?

Rachel needs a verified client identity and contact route, the tool's repository or documentation, evidence about first-party versus import provenance, current maintainer activity, and evidence that better recording-MBID coverage is a meaningful outreach issue. The ListenBrainz documentation shows why original-submission provenance can matter, while the MetaBrainz repository identifies public issue and community channels where Rachel can verify the label.

What could Rachel wrongly conclude?

Rachel could read rank 1 and score 1.000 as proof that listenbrainz is the highest-priority outreach client and immediately add it to the candidate list. That conclusion is too strong. The score establishes only that this exact label has the largest observed aggregate under the chosen filters. unknown does not mean importer, and it does not mean regular client; it means the label is in neither configured list. Rachel should hold the label for classification and ownership verification while retaining the ranking as an investigation signal.

Independent calculation

I independently read dataset_listenbrainz/sample/listens_20260904.csv, trimmed and case-folded client labels, counted event rows, counted distinct positive user_id values, counted affected accounts among rows without a syntactically valid UUID recording MBID, and applied the formula in outreach/filters.py.

```
listenbrainz listens 1090 accounts 765 affected 765 valid 0 missing 1090 type unknown
navidrome listens 889 accounts 719 affected 318 valid 469 missing 420 type regular
pano scrobbler listens 520 accounts 360 affected 360 valid 0 missing 520 type regular
web scrobbler listens 317 accounts 166 affected 166 valid 0 missing 317 type regular
listenbrainz android listens 305 accounts 211 affected 211 valid 0 missing 305 type regular
roon listens 86 accounts 63 affected 63 valid 0 missing 86 type regular
```

After importer exclusion and the 100-account floor, listenbrainz is the maximum on both ranking inputs: 765 affected accounts and 1,090 unusable IDs. At reach weight 0.5, both log-normalized terms equal 1, so its score is 0.5 × 1 + 0.5 × 1 = 1.000.

5. Verdict

Break for immediate outreach eligibility; keep as an investigation aid. The app is useful because it surfaces listenbrainz quickly: its observed volume and affected-account reach are large. It is not sufficient evidence to place the label directly on Rachel's outreach candidate list because the app has no verified classification or maintainer identity. Rachel is better off using the app to prioritize a verification step, then using source repositories, issue trackers, documentation, and community channels to decide whether outreach is appropriate.

Proposed change request

CR-09: Separate screening rank from verified outreach eligibility

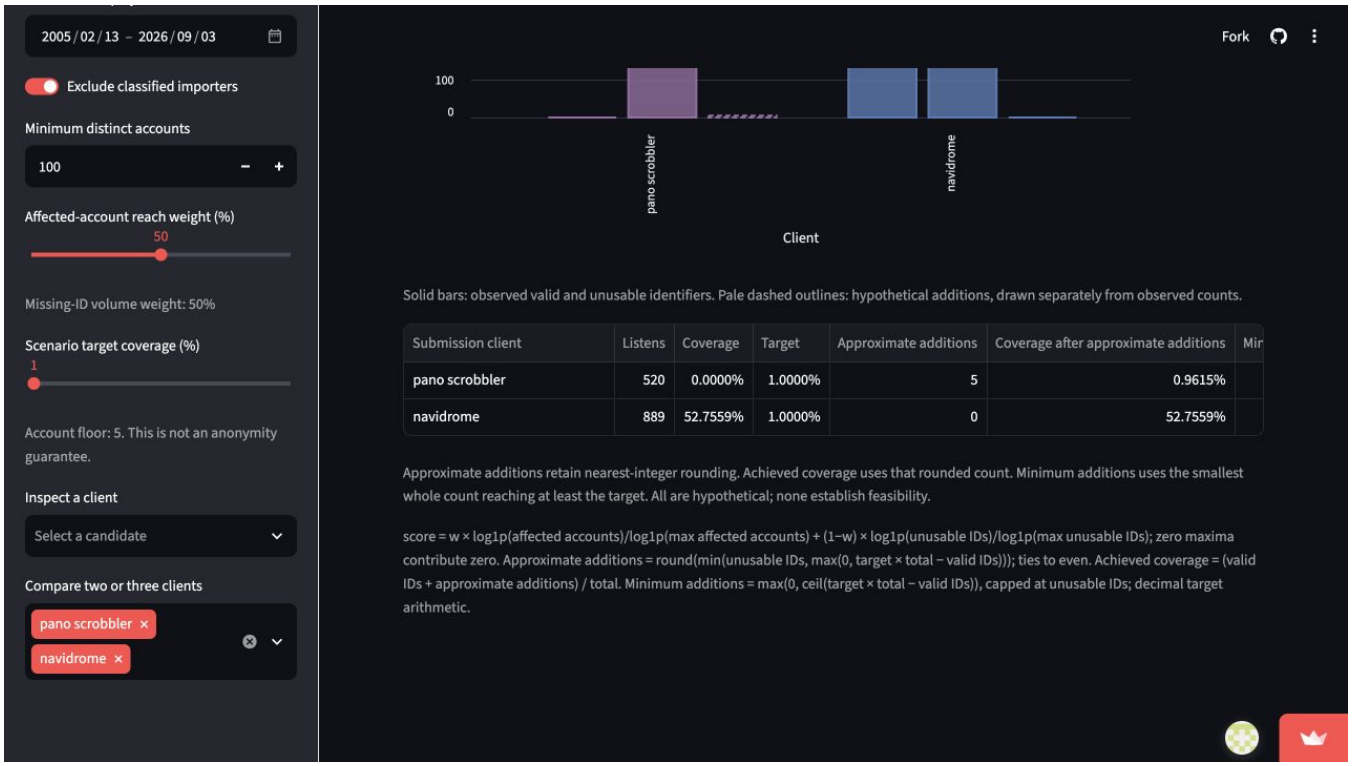
- From: Rachel case, 2026-11-08
- Part: Shortlist eligibility and client classification
- Evidence: In Sample Data mode with playback dates 2005-02-13 through 2026-09-03, classified-importer exclusion on, minimum accounts 100, and reach weight 50%, the independent sample calculation gives listenbrainz 1,090 listens, 765 distinct accounts, 765 affected accounts, 1,090 unusable IDs, and score 1.000. The configuration lists exact importer and regular-client names; listenbrainz is in neither list and is therefore classified unknown. The loader excludes only NOT_REPORTED and clients below the account floor before ranking; rank_clients does not require client_type == "regular".
- Change: Keep unknown labels visible as investigation leads, but add a clearly separate outreach-eligibility status or filter that requires verified classification and ownership. Label the current rank as screening priority and do not imply that rank alone makes an unknown client an outreach candidate.
- Priority: medium
- Status: open

Updated live outcome for Nora

With sample mode, UTC playback dates 2005-02-13 to 2026-09-03, importer exclusion on, minimum accounts 100, reach weight 50%, Pano and Navidrome selected, and target 1%, the repaired comparison reports Pano's approximate additions as 5, achieved coverage about 0.9615%, and minimum additions as 6. Navidrome already exceeds 1%, so both addition measures are zero. The complete comparison briefing now contains both clients even with no inspected client selected. The downloaded live CSV was opened and independently inspected: it contained exactly two client rows, the 1% target, sample mode, playback dates, 5 and 0 approximate additions, 6 and 0 minimum additions, and no account identifier column. It also preserves the common filters. It contains aggregate results rather than account identifiers.

The recommendation after repair is narrower and more useful: the app can document the arithmetic and a two-client comparison, but Nora must still investigate metadata pathways and maintainer feasibility before preferring an intervention. A minimum hypothetical count is not an estimate of a maintainer's impact. The fixes improve the briefing without turning the scenario into causal evidence.

Live evidence: Updated live scenario comparison, left side. Full screenshot includes settings.



Enlarged crop from the same live screenshot.

Submission client	Listens	Coverage	Target	Approximate additions	Coverage after approximate additions	Mir
pano scrobbler	520	0.0000%	1.0000%	5	0.9615%	
navidrome	889	52.7559%	1.0000%	0	52.7559%	

Live evidence: Same scenario table, scrolled right to show minimum additions. Full screenshot includes settings.

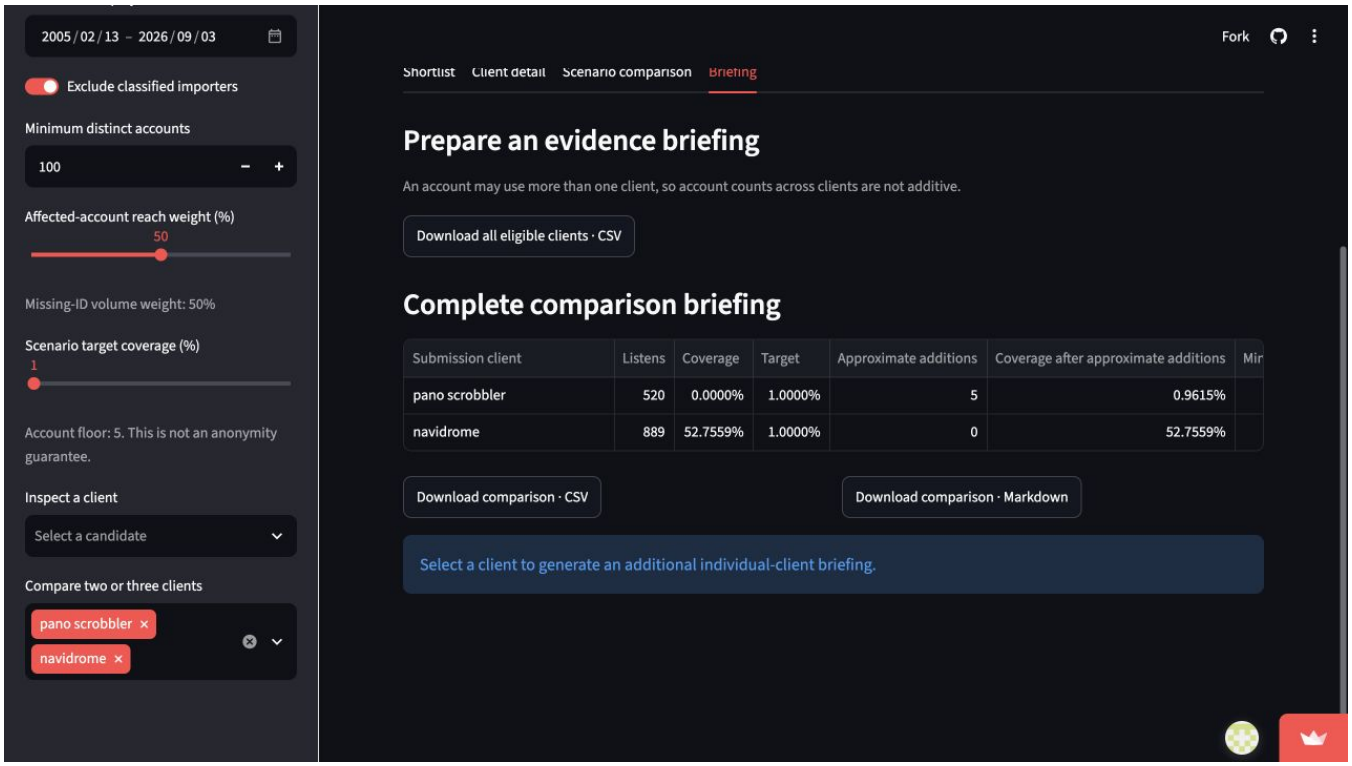


Listens	Coverage	Target	Approximate additions	Coverage after approximate additions	Minimum additions to reach target
520	0.0000%	1.0000%	5	0.9615%	6
889	52.7559%	1.0000%	0	52.7559%	0

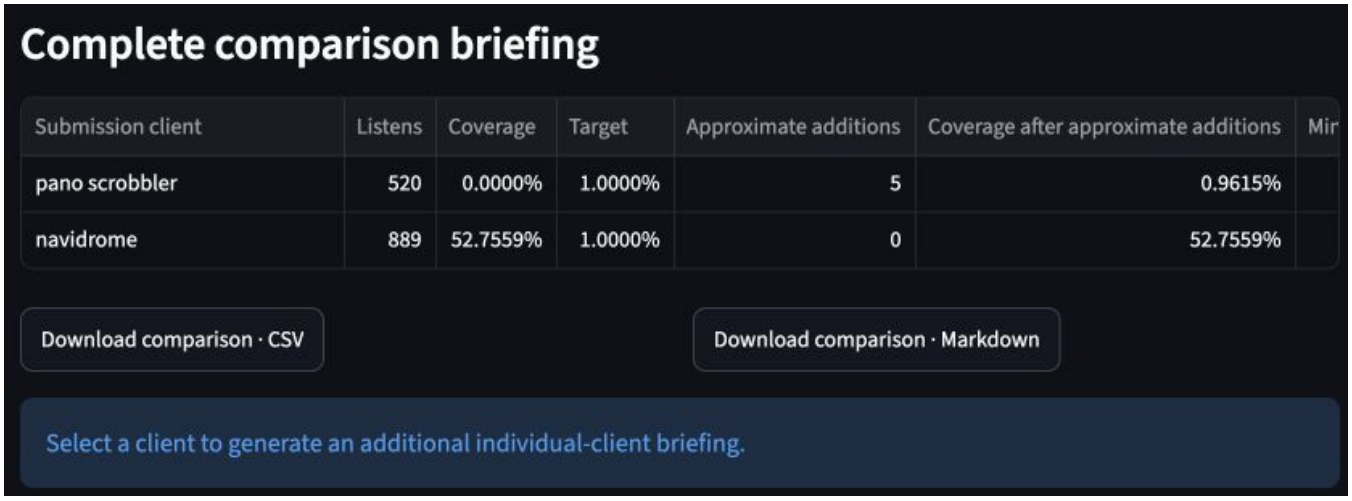
Enlarged crop from the same live screenshot.

Listens	Coverage	Target	Approximate additions	Coverage after approximate additions	Minimum additions to reach target
520	0.0000%	1.0000%	5	0.9615%	6
889	52.7559%	1.0000%	0	52.7559%	0

Live evidence: Updated complete comparison briefing. Full screenshot includes settings.



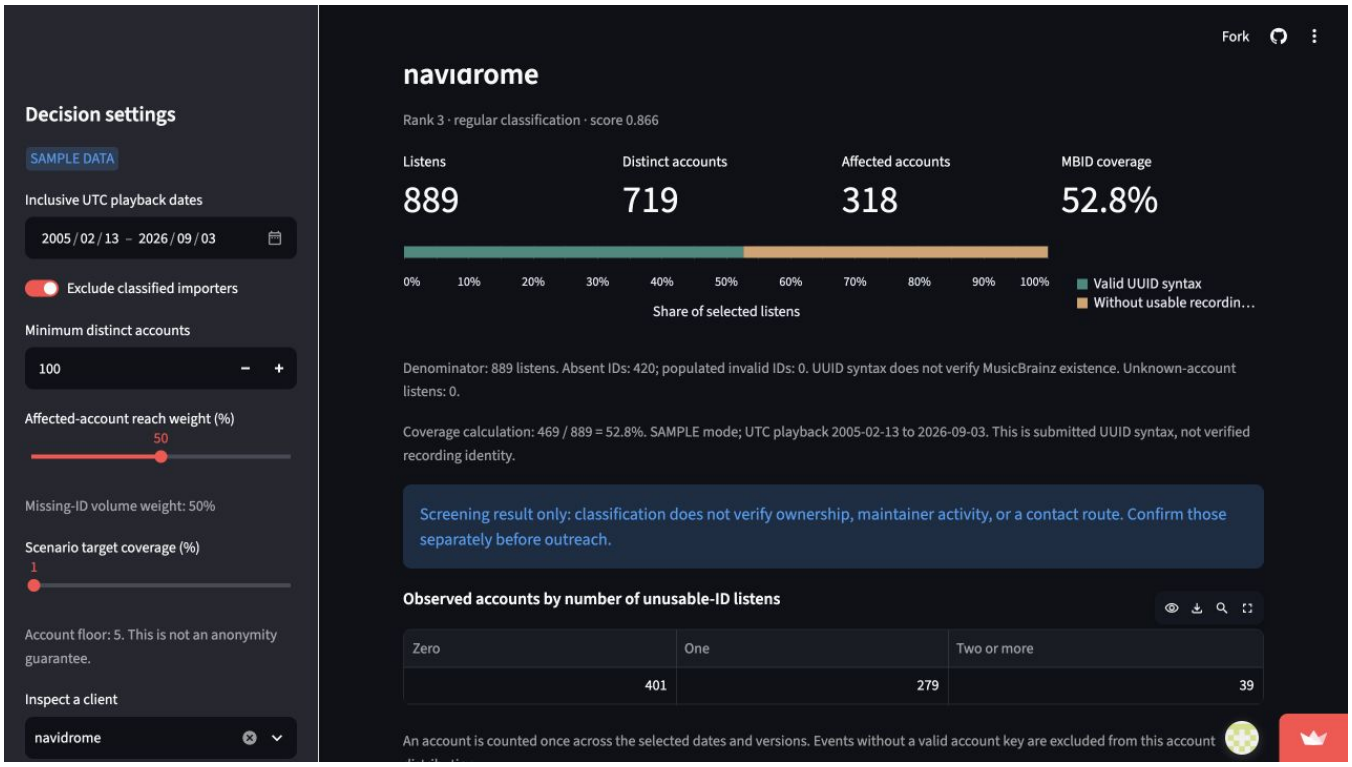
Enlarged crop from the same live screenshot.



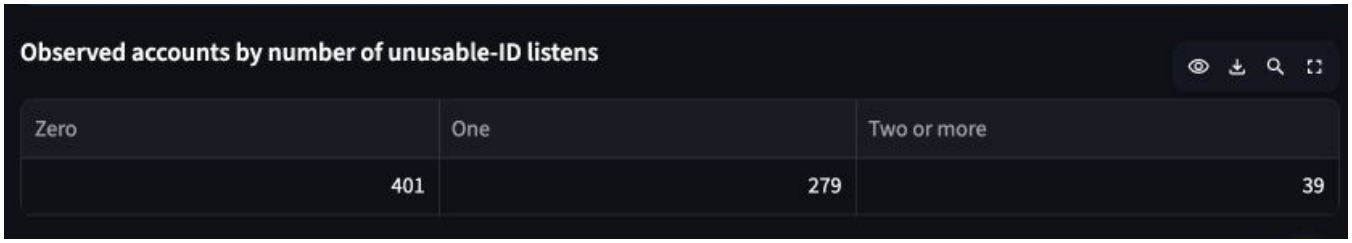
Updated outcomes from the other cases

Alex can inspect the new version-gap table sorted by unusable-ID count and see the valid-ID numerator, denominator, computed coverage and sample/date scope together. Morgan can use the same reported-version evidence as an investigation lead, while still consulting actual support requests, release history and operating-system evidence before making a support decision. Ana can compare all eligible client ranks at 0%, 50% and 100% reach weight beside their affected-account and unusable-event counts. Independent counting reproduces the Android/Web reversal in her case. Rachel sees that the list is a screening rank and that outreach status is **Not verified** for every label: the dataset contains no verified ownership directory. No row is labeled outreach-ready merely because it is classified regular.

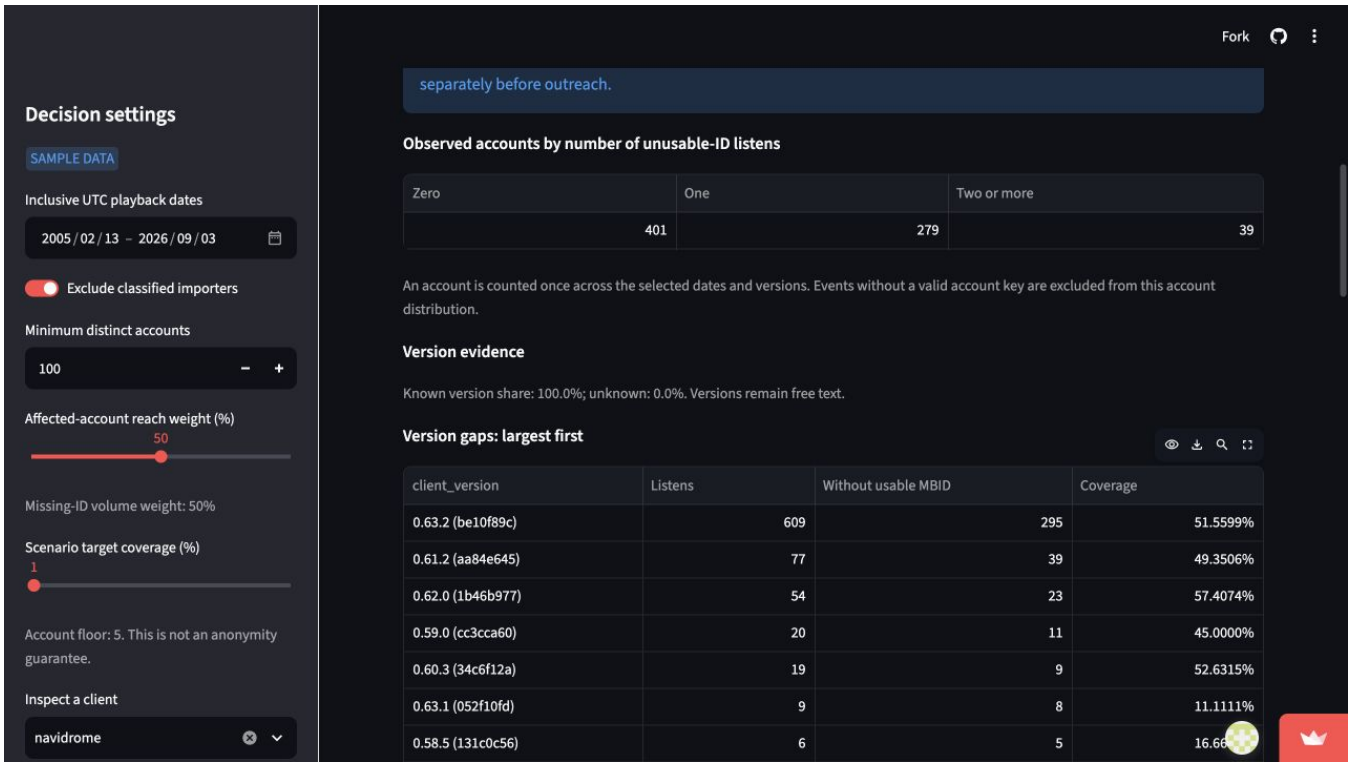
Live evidence: Live Navidrome detail and account distribution. Full screenshot includes settings.



Enlarged crop from the same live screenshot.



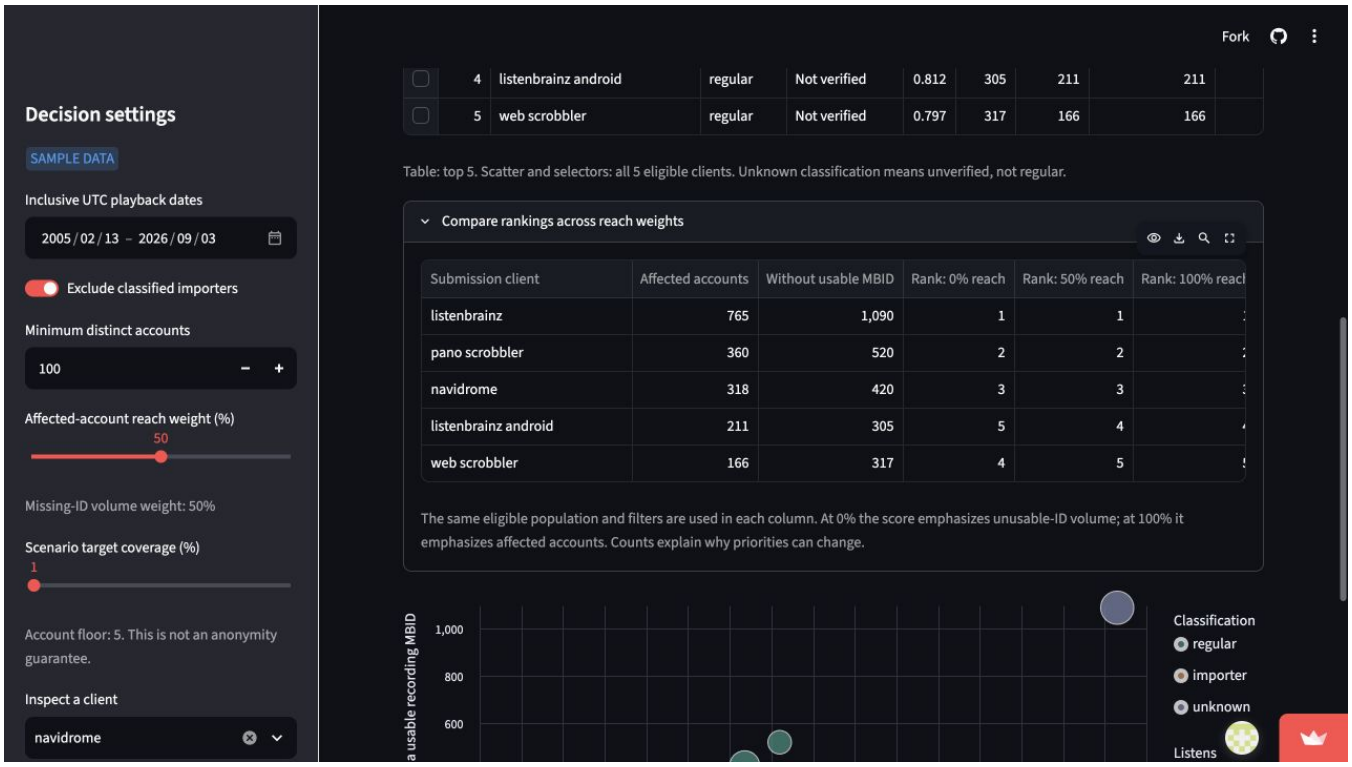
Live evidence: Live Navidrome version gaps sorted by missing IDs. Full screenshot includes settings.



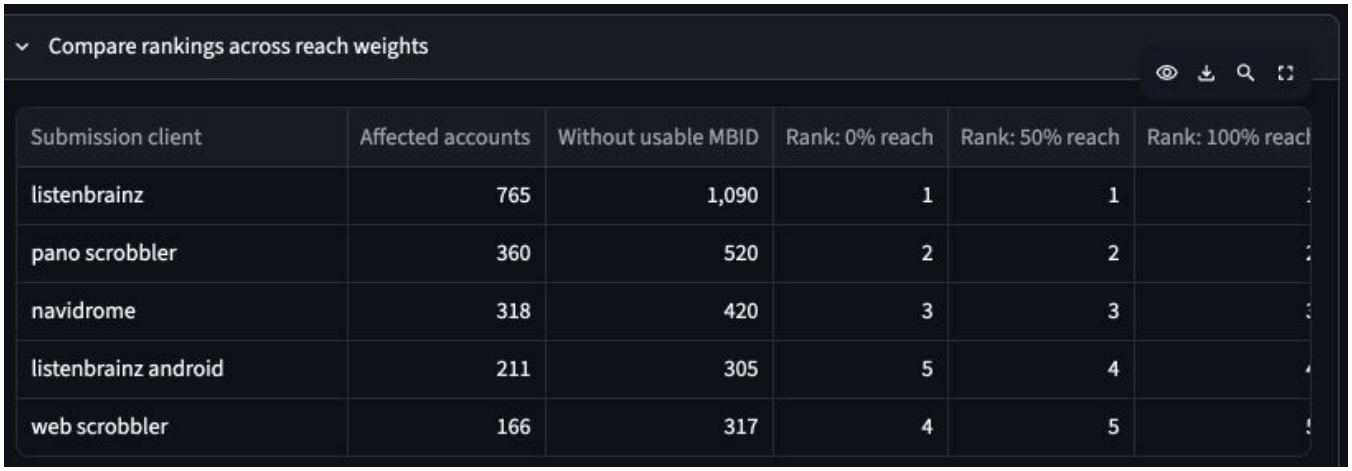
Enlarged crop from the same live screenshot.



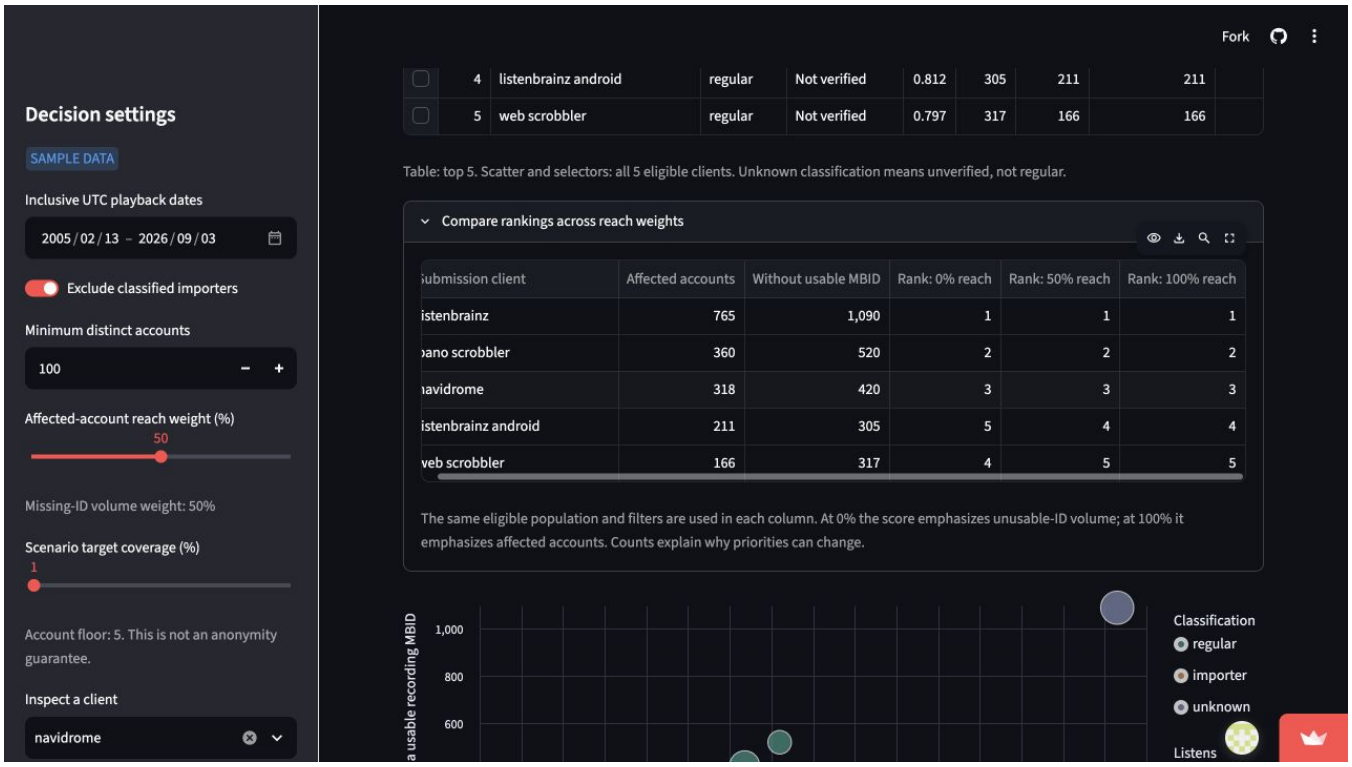
Live evidence: Live ranking comparison, left side. Full screenshot includes settings.



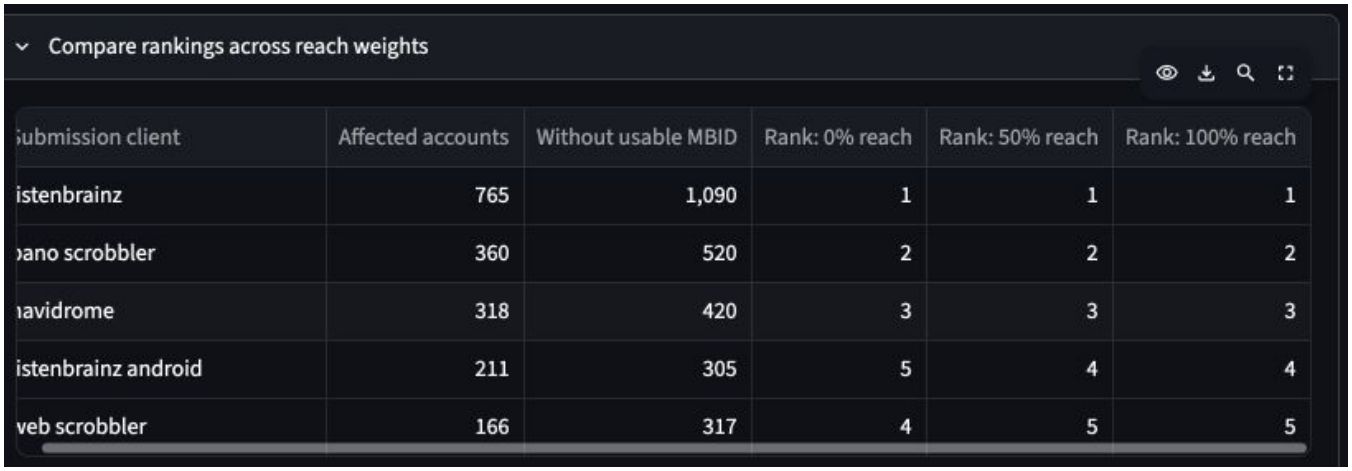
Enlarged crop from the same live screenshot.



Live evidence: Live ranking comparison, scrolled right to show the 100% reach column. Full screenshot includes settings.



Enlarged crop from the same live screenshot.

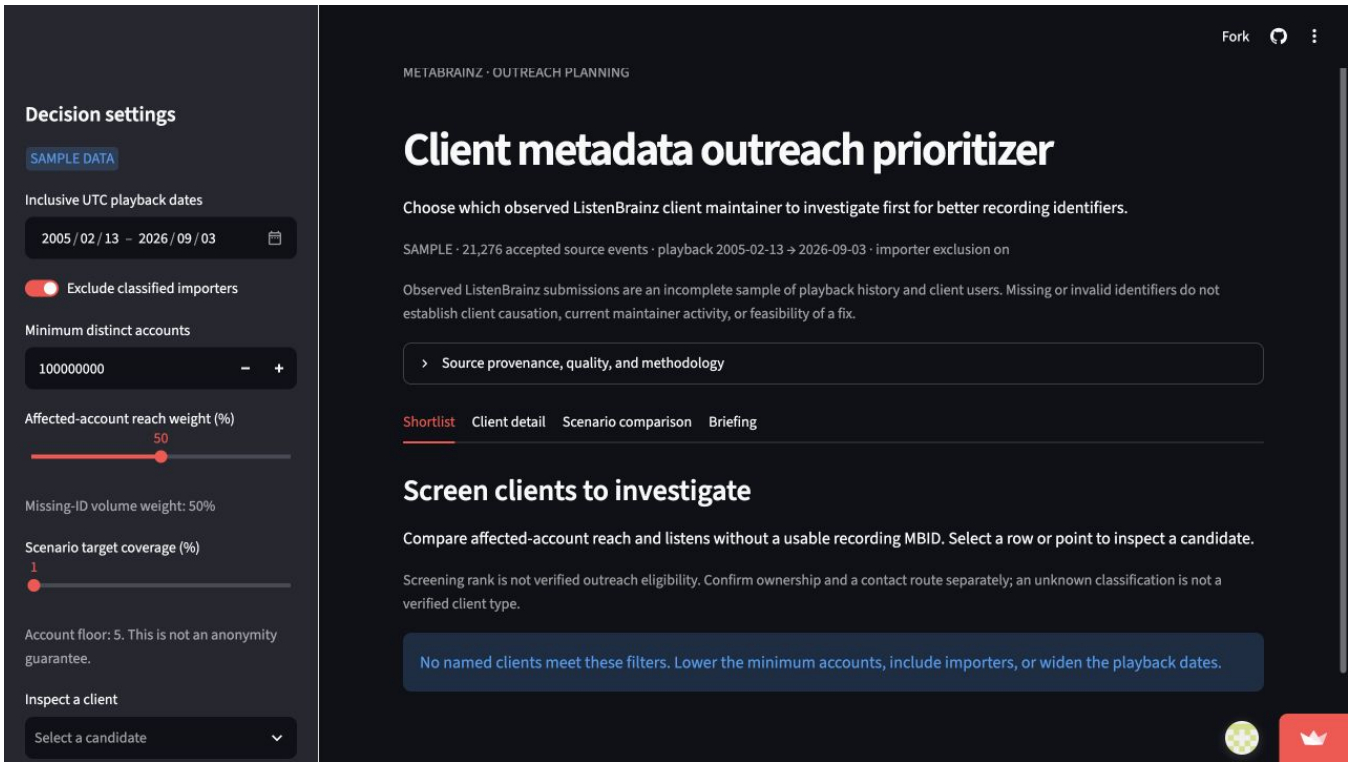


The new account-distribution view directly addresses CR-08. For Navidrome, an independent CSV grouping yields 401 observed accounts with zero unusable-ID listens, 279 with one and 39 with two or more. These sum to 719 distinct accounts; 279 + 39 = 318 affected accounts. This does not represent the full client user population. The aggregate is computed once per account across the selected date range and versions, and the temporary account keys are not displayed or exported.

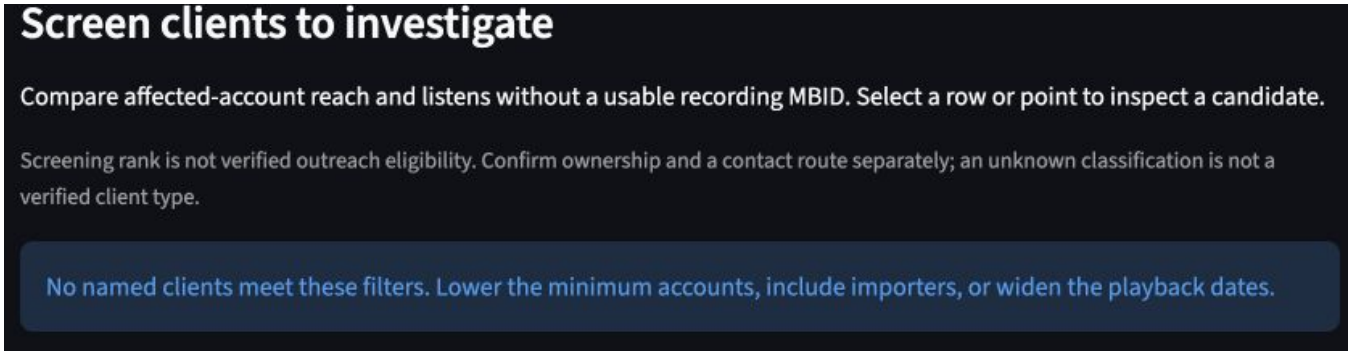
Additional workflow checks

The default screen opens with 5 eligible clients and no preselected client. Raising the account minimum to an extreme value returns an explanatory empty result and clears stale selections. Restoring the minimum recovers the shortlist. A two-client briefing can be generated without selecting an inspected client. Source scope, unknown classification and hypothetical-scenario cautions remain visible. These checks concern workflow robustness; they do not substitute for the missing member cases.

Live evidence: Live empty-result recovery check. Full screenshot includes settings.



Enlarged crop from the same live screenshot.



3. Numbers

Definitions and counting population

One source row is a submitted listen. Preparation groups accepted events by UTC playback date, normalized submission client, version, internal account key and classification. One prepared row is therefore a group of events, not one unique person. Account identifiers remain private internal keys; public outputs contain only aggregate counts.

Measure	Definition and units	Correctness condition
Total listens	Accepted event count in selected playback dates and filters	Matches direct source counting; valid + absent + invalid equals total.
Submitted-MBID coverage	Syntactically valid recording-UUID events / all selected events	Same denominator as total; not an identity/existence check.
Distinct accounts	Unique valid account keys across the entire selected client/date set	A repeated account counts once per group, not once per day or version.

Measure	Definition and units	Correctness condition
Affected accounts	Unique valid accounts with at least one absent/invalid-ID event	At most distinct accounts; counts across clients are not additive.
Unusable-ID volume	Absent plus populated-invalid submitted recording IDs	Complements syntactically valid events; does not measure missing server mappings.
Score	w log(1+affected)/log(1+max affected) + (1-w) log(1+unusable)/log(1+max unusable)	Maxima use all eligible clients; a zero maximum contributes zero; sort by score, gap volume, then name.
Approximate additions	Nearest integer to min(unusable, max(0, target × total - valid)); ties to even	Preserves the documented original scenario; may fall below the target.
Achieved coverage	(valid + approximate additions) / total	Describes the rounded hypothetical count, not observed improvement.
Minimum additions	Smallest nonnegative integer reaching at least target coverage, capped at unusable events	Exact decimal target arithmetic avoids a false extra addition at integer boundaries.

A client must be named and meet the chosen distinct-account threshold after filters. Importer exclusion removes exactly classified importer events; unknown labels remain unverified. Invalid account values contribute listens but no reach. Versions inherit parent eligibility; their account counts must not be added across versions to recover client reach.

Independent audit and measured results

Run `uv run python scripts/validate_mvp.py` from the repository root. This audit independently reads the portable CSV using Python counters and sets, rather than the production adapters, groupings or rank function. It compares the app against those direct counters. Rank expectations use a separately written natural-log calculation; scenario thresholds use exact fractions and an exhaustive integer search. The report is saved in `evidence/mvp-numbers.json`.

Source sample measure — all events, before eligibility filters	Independent result
Events	21,276
Syntactically valid recording IDs	654
Absent IDs	20,622
Populated-invalid IDs	0
Distinct accounts	4,060
Affected accounts	3,546
Submitted-ID coverage	3.073886%

The default eligible-client population is different: 5 clients, 3,121 observed listens and 2,652 unusable-ID events. Do not compare its coverage denominator with the whole-source percentage above. The precise Pano/Navidrome values and the independent 1% calculation appear in the included case. The sample-source SHA-256 is `ac66335410c79c991c3b9609bbe0829789399be056652626119a5497bc84f0ee`.

Independent comparison	Scope	Result
Client metrics and rank	Default filters, 5 eligible clients	PASS

Independent comparison	Scope	Result
Account distribution and version gaps	All 5 default eligible clients, including Navidrome 401 / 279 / 39	PASS
All-client ranking comparison	0%, 50%, and 100% reach; independent score ordering	PASS
Client metrics and rank	Importers included, minimum accounts 5, 27 eligible clients	PASS
Client metrics and rank	Single playback day 2026-09-03, 5 eligible clients	PASS
Empty threshold	Minimum accounts 100,000,000, 0 eligible clients	PASS
Version and measured-week metrics	Every default eligible client	PASS
Scenario counts and threshold minimality	Every default eligible client at 1%, 75%, 100%	PASS

Portable-data parity was also checked against the original course sample: every event and measurement field is retained, with a one-to-one account replacement. No inverse lookup is packaged. The source identifier transformation changes source/preparation fingerprints but does not change equality-based counts.

The original live table displayed a 75% target as 70% because percentage precision was too coarse; the table formatting was corrected. Nora's historical screenshot shows Navidrome coverage as 52.7%, while exact counts give 469/889 = 52.7559%; the current table uses finer precision. These are display precision issues, not unexplained differences in event totals. Full-data figures and historical timing claims from earlier build reports are not presented as newly measured public-app results.

4. Tests

What the suite protects

The suite uses hand-checkable fixtures, public-sample UI flows, independent audits and a simulated deployment update. The final clean-environment run is recorded in `evidence/test-summary.json`: **31 passed, 1 skipped, 0 failures**. It used Python 3.14.7 and the new `uv.lock` environment. The skipped integration test requires a separately prepared full dataset; the deployed MVP uses the bundled sample. Full-data deployment and arbitrary concurrency are not claimed.

Risk or property	Test evidence	Why it matters
Wrong rows, units or date boundaries	January fixture; single-day and cross-month checks; raw/CSV adapter parity	A correct-looking chart can otherwise answer the wrong date/population question.
Double-counted reach	Shared accounts across clients/versions; forced spill equals in-memory results	Event totals can be added, but distinct people cannot be recovered by summing subgroup counts.
Reconciliation and bad inputs	Missing/invalid ID split; malformed JSON/timestamps; unsupported schema; duplicate inputs	Defective inputs must be counted or rejected explicitly.
Cache validity and interrupted preparation	Changed input/rule checks; failed write retains old completed cache	Reusing stale or half-written data can silently change decisions.
Ranking and scenario boundaries	Zero maxima, ties, invalid targets; exact fixture counts	Rankings must stay deterministic and estimates remain bounded.

Risk or property	Test evidence	Why it matters
Honest milestone interpretation	New exact-fraction regression: 1%, 7%, 50%, 75%, 100% on multiple totals; one fewer addition cannot reach the target	Protects the difference between an approximation and a threshold, including decimal boundaries.
Account distribution and all-client sensitivity	New fixture checks plus an independent per-account CSV grouping, and a live-sample UI check for the 401 / 279 / 39 split and Android/Web rank swap	The observed reach distribution and rank changes must be exact and understandable.
Complete and private exports	New two-client CSV/Markdown test; source/filter fields; injected private sentinel discarded	The exported evidence must support the comparison without carrying internal identifiers.
Usable comparison workflow	New two-client briefing test with no inspected client; individual-client briefing retained	A user should not lose the comparison when preparing a recommendation.
Deployment update recovery	New isolated launcher test changes the source timestamp and reruns	A repository refresh must trigger valid preparation rather than leave the public app in an input-changed error state.
UI edge states	Real bundled sample; extreme account minimum; empty and selection states; no raw-data reads during control changes	Common interactions must work from the packaged repository.

The seven MVP-specific regression tests are in `outreach-app/tests/test_mvp_regressions.py`. The account-distribution audit and all-client ranking cross-check are in `scripts/validate_team_findings.py`, with aggregate results in `evidence/team-findings.json`. Two older UI tests were repaired to prepare and use the bundled sample rather than depending on a neighboring dataset/cache; that fixture repair is distinguished from new test coverage. Existing Module 10 gap tests were retained. Historical starting-build tests are described in `BUILD_FINDINGS.md`; they are not credited as new MVP work.

Reproduce and interpret the evidence

From the repository root, run `uv sync --locked`, then `uv run python scripts/validate_mvp.py` and `uv run python scripts/validate_team_findings.py`. For the test suite, change into `outreach-app` and run `uv run python -m pytest -q`. The ZIP excludes `.venv`, private prepared data, secrets, raw archives and caches. A recipient can recreate the environment and sample preparation from committed files.

The live checks and unit/integration results establish the tested sample behavior and key arithmetic; they do not establish current maintainer activity, UUID semantic accuracy, causal effects, arbitrary data-size limits, Windows execution or high-concurrency performance. Before submission, add the completed unlisted demo video link, open both links without sign-in, and rebuild the ZIP from the final reviewed commit. Only the final submitted package should be graded as the completed MVP.